



Diogo Silva
Duarte Santos
Gabriel Janicas
Miguel Reis
Rodrigo Moço
Tiago Garcia

Secure Environment for Academic Assessment



**Diogo Silva
Duarte Santos
Gabriel Janicas
Miguel Reis
Rodrigo Moço
Tiago Garcia**

Secure Environment for Academic Assessment

Este projeto foi concebido e desenvolvido no âmbito da Unidade Curricular Projeto em Engenharia de Computadores e Informática, realizada sob a orientação científica de João Rafael Duarte de Almeida, Professor auxiliar Departamento de Eletrónica, Telecomunicações e Informática da Universidade de Aveiro, do Doutor João Paulo Barraca, Professor associado do Departamento de Eletrónica, Telecomunicações e Informática da Universidade de Aveiro.

Regente / Professor

Prof. Doutor José Moreira
professor auxiliar da Universidade de Aveiro

Docente / Professor

Prof. Doutor Rui Luis Andrade Aguiar
professor catedrático da Universidade de Aveiro

palavras-chave

seguro, avaliação, portátil, monitorização, integridade.

resumo

A integridade e fiabilidade dos sistemas de avaliações académicos são atualmente um dos maiores desafios que muitas instituições enfrentam. A necessidade de um método seguro e eficaz para gerir processos de avaliação, proteger dados sensíveis e garantir a igualdade entre membros da comunidade académica nos novos métodos de avaliação digitais torna-se cada vez mais importante.

Este projeto visa ao desenvolvimento de um dispositivo portátil, juntamente com uma webapp para facilitar o processo de criação e realização de avaliações. Garantindo um método seguro em que os estudantes possam utilizar os periféricos dos seus computadores pessoais num ambiente isolado.

Apesar de diversas soluções terem sido criadas com o propósito de melhorar este processo, ainda nenhuma mostrou ser eficaz e adequada aos requisitos dos docentes. Numa primeira fase deste trabalho foi feita um estudo dos sistemas existentes para identificar possíveis vantagens e desvantagens do seu uso, este estudo associado à análise do problema permitiu identificar os principais requisitos do projeto. No seguimento do mesmo, foi concebida a arquitetura e reconhecimento de possíveis desafios de implementação, incluindo questões relacionadas à segurança, desempenho e usabilidade.

keywords

secure, evaluation, portable, monitoring, integrity.

abstract

The integrity and reliability of academic assessment systems is currently one of the biggest challenges that many institutions face. The need for a secure and effective method to manage evaluation processes, protect sensitive data and guarantee fairness among members of the academic community in new digital evaluation methods is becoming increasingly important.

This project aims to develop a portable device together with a webapp to facilitate the process of creating and carrying out evaluations. Ensuring a secure method in which students can use the peripherals of their personal computers in an isolated environment.

Although various solutions have been created with the aim of improving this process, none have yet proved to be effective and suitable for professors' requirements.

In the first phase of this work, a study of existing systems was carried out to identify possible advantages and disadvantages of their use, this study, combined with the problem analysis, made it possible to identify the main requirements of the project. This was followed by designing the architecture and identifying possible implementation challenges, including issues related to security, performance and usability.

**acknowledgement of use of
AI tools**

**Recognition of the use of generative Artificial Intelligence
technologies and tools, software and other support tools.**

We acknowledge the use of ChatGPT 3.5 (OpenAI, <https://chat.openai.com>) to improve some sentences in terms of form, punctuation and vocabulary.

We also use DeepL Write (<https://www.deepl.com/en/write>) to enhance the grammatical construction and phrasing of the report.

Contents

Contents	i
List of Figures	v
List of Tables	vi
Glossary	vii
1 Introduction	1
1.1 Context	1
1.2 Motivation	2
1.3 Goals	2
1.4 Scope	3
2 State of the art	4
2.1 Related Work	4
2.1.1 Online Proctors	4
2.1.2 Transforming Exams (E-Exams in Australia)	5
2.1.3 SEB - Safe Exam Browser	6
2.1.4 Exam Control	6
2.1.5 EEL - Eletronic Exam Lab	7
2.2 Technologies used	9
2.2.1 Isolated System	9
2.2.2 Server	9
3 Conceptual Model and Architecture	11
3.1 Conceptual Model	11
3.1.1 Functional Requirements	11
3.1.2 Non-Functional Requirements	14
3.1.3 Actors	15
3.1.4 Uses Cases	16

3.1.5	Abuse Case	21
3.2	Architecture	23
3.2.1	Components	23
3.2.2	Workflows	25
3.2.3	Interaction Diagrams	28
4	Implementation	32
4.1	Server	32
4.1.1	API	32
4.1.2	Database	34
4.1.3	File Manager	35
4.1.4	Logs Service	36
4.2	WebApp	38
4.2.1	Architectural Overview	38
4.2.2	Middleware	38
4.2.3	Data-Access layer	38
4.2.4	Authentication and Authorization	39
4.2.5	UI Composition and Styling	39
4.2.6	Development Workflow and Tooling	39
4.3	DevApp	40
4.3.1	Architectural Overview	40
4.3.2	UI: App Pages	41
4.3.3	Subscriber	41
4.3.4	Logging	42
4.3.5	Modus Operandi	44
4.3.6	Build and Deployment	47
4.4	Operating System	48
4.4.1	Why we use an operating system	48
4.4.2	Live-Build	48
4.4.3	How it works - Live-Build	50
4.4.4	Internal structure of our image	51
4.4.5	Hardware compatibility	51
4.5	Raspberry PI's	53
4.5.1	Raspberrys Setup	53
4.5.2	Gadget Mode	54
4.5.3	Gadget Mode configuration	55
5	Security	57

5.1	Authentication and Authorization	57
5.1.1	JWT Implementation	57
5.1.2	Web Application Authentication	58
5.1.3	Device Authentication	58
5.1.4	Authorization Framework	60
5.1.5	Password Security	60
5.2	Communications Security	63
5.2.1	Transport Layer Security	63
5.2.2	API Communication Security	63
5.2.3	Real-time Communication Security	63
5.2.4	Database Communication Security	64
5.2.5	Redis Communication Security	64
5.3	Storage	65
5.3.1	File Storage Security	65
5.4	Operating System Security	69
5.4.1	Internet Access Control	69
5.4.2	User and Privilege Management	70
5.5	Raspberry vs USB Pendrive	72
5.5.1	Active Element vs Passive Element: Fundamental Differences:	72
5.5.2	Raspberry vs USB Pendrive	73
5.5.3	Conclusion	74
6	Tests and Results	75
6.1	Tests	75
6.2	Results	75
6.2.1	Secure and Controlled Exam Environment	75
6.2.2	Real-Time Monitoring and Support	76
6.2.3	Usability and Comfort	76
6.2.4	Portability and Affordability	76
6.2.5	Simplified Exam Management for Teachers	76
7	Future Work	77
7.1	Secure Real-Time Student Screen-sharing	77
7.1.1	Objectives	77
7.2	Fraudulent-Activity Alerting Framework	77
7.2.1	Objectives	78
7.3	Plug-gable Authentication via External Identity Providers	78
7.3.1	Objectives	78

7.4	BIOS-Level Bootloader Integration	78
7.4.1	Objectives	78
7.5	Centralized, Remote Device Management Console	78
7.5.1	Objectives	79
Referências		80
A Additional content		80
A.1	Live-Build Config	80
A.2	Whitelist.py	82
A.3	Endpoints Tree	84
A.4	Docker Compose Configuration	85

List of Figures

2.1	EEL Components Diagram.	8
3.1	Use Case Diagram for Professor.	16
3.2	Use Case Diagram for Student.	18
3.3	Use Case Diagram for Admin.	19
3.4	Use Case Diagram for Auditor.	20
3.5	Architecture Diagram.	23
3.6	Activity Diagram - Exam Creation.	26
3.7	Activity Diagram - Exam Solving.	27
3.8	Interaction Diagram - Evaluation Process.	29
3.9	Interaction Diagram - Direct Device Configuration.	30
3.10	Interaction Diagram - Remote Configuration.	31
4.1	Database ERD.	35

List of Tables

3.1	Functional Requirements - WebApp.	12
3.2	Functional Requirements - Isolated System.	13
3.3	Functional Requirements - Server.	13
3.4	Non-Functional Requirements.	14
4.1	Approach - Isolated System Image	49
4.2	Customization - Isolated System Image	49
4.3	Result - Isolated System Image	50
4.4	Gadget Mode	54
5.1	Raspberry Pi vs USB Pendrive - Security Level	73

Glossary

AES	Advanced Encryption Standard	MIME	Multipurpose Internet Mail Extension
AI	Artificial Intelligence	ORM	Object-Relational Mapping
API	Application Programming Interface	OS	Operating System
ASCII	American Standard Code for Information Interchange	PIE	Portable Isolated Environment
BIOS	Basic Input/Output System	PXE	Preboot Execution Environment
CORS	Cross-Origin Resource Sharing	RBAC	Role-Based Access Control
CSRF	Cross-Site Request Forgery	SEB	Safe Exam Browser
CU	Curricular Units	SHA	Secure Hash Algorithm
DETI	Departamento de Eletrónica, Telecomunicações e Informática	SQL	Structured Query Language
DLL	Dynamic Link Library	SSE	Server-Sent Events
DTLS	Datagram Transport Layer Security	SSL	Secure Sockets Layer
DWC2	DesignWare Controller 2	TLS	Transport Layer Security
EEL	Eletronic Exam Lab	UA	Universidade de Aveiro
ELF	Executable and Linkable Format	UEFI	Unified Extensible Firmware Interface
ERD	Entity-Relationship Diagram	UI	User Interface
GCM	Galois/Counter Mode	USB	Universal Serial Bus
HMAC	Hash-based Message Authentication Code	WebRTC	Web Real-Time Communication
HTTP	Hypertext Transfer Protocol	XSS	Cross-Site Scripting
HTTPS	Hypertext Transfer Protocol Secure	VM	Virtual Machine
IdP	Identity Provider	URL	Uniform Resource Locator
JWT	JSON Web Token	CA	Certificate Authority
LMS	Learning Management System	SQL	Structured Query Language
		DOS	Disk Operating System

Introduction

Evaluation is a main component of the academic world, serving as a structured means of measuring the student's knowledge in a specific area and what was learnt throughout their studies. It assesses knowledge retention, critical thinking and application skills. It provides feedback to students and professors on individual progress, helping identify areas for improvement, while also revealing where additional support may be needed to achieve academic goals. Evaluation also plays a vital role in maintaining academic standards, since the students are the direct result of an institution's work and effort.

One significant challenge all institutions encounter is the prevalence of academic cheating during said evaluations, which undermines the integrity and reliability of the evaluation process. When students engage in dishonest practices, it devalues the knowledge and skills of their peers and affects the overall credibility of an institution. To maintain a fair and equitable environment it is crucial to implement effective measures to prevent cheating.

There are several types of evaluation, and they have been evolving with the technological developments and necessities of the modern days. From simple paper exams to more interactive and digital ones, the methods used to evaluate the students' knowledge have transformed significantly. Currently, institutions tend to rely on digital exams to facilitate the distribution process and provide students with access to recent and practical tools for preparation in a real-world context. However, this reliance on digital examinations presents a challenge in maintaining academic standards and ensuring the integrity of the evaluation process.

1.1 CONTEXT

This project was designed and developed as part of the Project in Computer and Informatics Engineering course at the University of Aveiro. The project addresses a significant challenge faced by the academic community, namely the need for a secure and controlled environment for digital evaluations. With institutions increasingly adopting real-life scenarios facilitated by the digital format, ensuring the integrity and fairness of evaluations has become a critical priority. This solution introduces a portable, dedicated device that plugs into the student's

laptop to create an isolated and secure examination environment, thereby enabling fair and reliable evaluation practices.

1.2 MOTIVATION

Currently, the evaluation process at most institutions relies on laboratory computers to create controlled environments. Although this is one of the best solutions, it requires significant resource management, not only in terms of the number of computers available, but also in terms of the human resources required to maintain and monitor these setups. The reliance on physical space and specific equipment can create logistical challenges, especially during high demand testing periods. Another important aspect to consider is the unfamiliarity and discomfort that students may experience when using these devices, which can negatively affect their concentration and performance during the exam. Many students are used to their own personal setups, and the transition to a different environment with unfamiliar peripherals adds unnecessary challenges to an already demanding process. In addition to issues such as the comfort and resource demands of the institutions, we also have to take into account the fact that certain software lacks security, such as Safe Exam Browser (SEB), which solves the problems mentioned above but nowadays ways have been found to bypass the software's security. Despite the existence of software created to address this problem, it falls short in the capability of delivering a safe and reliable environment. As the need for a solution increases, the pressing need for a reliable, secure and affordable device becomes apparent. By combining the portability and security of a device that connects directly to the user's familiar environment, a solution is created that aims to address the current limitations of alternative solutions.

1.3 GOALS

The main objective of this project is to design and implement a secure, user-friendly, and efficient system for conducting digital examinations in academic settings. This system is intended to address the limitations of existing solutions by integrating a dedicated, portable device with a web application and server, thereby ensuring reliability, scalability, and ease of use. The specific goals of this project are as follows:

- **Secure and Controlled Exam Environment:** Create an isolated environment for students that restricts access to non-essential resources and ensures academic integrity during exam taking. This will include measures such as blocking external connections and safeguarding personal data from unauthorized access or malware.
- **Real-Time Monitoring and Support:** Enable teachers to monitor students' progress and activity in real-time, allowing for on-time intervention in cases of technical issues or rule violations.
- **Usability and Comfort:** Provide an intuitive and accessible system interface for students. The solution will allow students to use familiar peripherals and offer useful features as time tracking.

- **Portability and Affordability:** Develop a portable and affordable device that reduces dependence on institution infrastructure and laboratories and also mass usage.
- **Simplified Exam Management for Teachers:** Equip teachers with tools to easily configure exams, upload and download exam data, and conclude exam sessions efficiently, ensuring all students follow the same schedule and rules.

1.4 SCOPE

The development of the project focuses only solely on in-classroom use, prioritizing a safe and controlled application context. When the use of the secure environment is requested by the university's teaching staff, the devices associated with its use are provided by the respective university teaching team and collected at the end of each use. This equipment is delivered at the beginning of each user session and collected soon after it ends, ensuring that it remains under continuous supervision and control.

Functionalities related to non-presential activities, such as remote access or extracurricular use, are outside the scope of the project, reinforcing the objective of keeping the system restricted to the face-to-face and controlled environment within the university.

Additionally, this approach minimizes potential vulnerabilities associated with the use of devices in less controlled contexts, such as external networks or personal settings, aligning with the principles of digital security and protection of sensitive data. With this, it is ensured that the system is used efficiently and reliably, promoting a teaching-learning experience appropriate to the educational environment.

State of the art

2.1 RELATED WORK

In this section, we review and explore existing research and implementations relevant to our topic. The goal is to contextualize our work by exploring prior contributions, methodologies and technologies that have been used in the past for similar challenges.

This review highlight gaps in current solutions, demonstrating how our work will diverge from the existing state-of-the-art. For instance, many existing approaches to online proctoring prioritize detection accuracy but fail to adequately address privacy concerns, leaving room for improvement in the ethical handling of user data. Furthermore, scalability remains a challenge, as most solutions are not optimized for diverse environments or large-scale deployment. After that approach, we aim to establish a clear understanding of the progress made in this area and how our work will contribute for the broader acknowledgement field.

2.1.1 Online Proctors

Description

Proctor is an academic integrity solution for personalized learning, developed by Verificent Technologies. It provides online proctoring and remote monitoring capabilities, leveraging Artificial Intelligence (AI) surveillance and continuous verification to ensure trusted exam integrity.

This solution is widely used across various educational environments, offering accessible proctoring for both high-stakes and low-stakes exams. Its standout feature is the ability to support personalized learning for each student, moving away from a one-size-fits-all approach to education.

Takeouts

Proctor offers a robust system to ensure the integrity of academic assessments. Its AI-driven, real-time examination monitoring functionality is a distinctive feature that continuously supervises each evaluation. By offering support for both high-stakes and low-stakes scenarios

and by facilitating customized, personalized learning experiences, it surpasses the limitations of a one-size-fits-all approach.

However, this capacity can also act as a hindrance. The act of maintaining genuine, real-time oversight necessitates the allocation of a dedicated proctor to each student, a process that rapidly becomes resource-intensive and undermines scalability. Furthermore, Proctor’s paid licensing model, which operates exclusively at the application level without encompassing system-wide control, may present additional financial and technical challenges for certain institutions.

2.1.2 Transforming Exams (E-Exams in Australia)

Description

The E-Exam [1] is a system developed in Australia to manage exams in educational institutions, offering both online and offline modes of operation. It serves as a plug-and-play alternative to traditional methods that require prior software installation. The process of connecting a USB stick to a computer and subsequently initiating the boot sequence from the device establishes a secure environment that imposes restrictions on student access to specific folders.

In online mode, exams are delivered via Moodle, with the USB stick serving as a backup solution to ensure data integrity, even in cases of system crashes or internet outages. In offline mode, the system downloads exam documents from a server and stores them in a specific directory, allowing students to complete their exams. Both modes incorporate automatic periodic backups to safeguard progress.

This approach is considered one of the safest available, as it prevents access to personal directories and browsing on unsecured Wi-Fi networks. However, it is limited to supporting specific file types associated with Google Workspace, such as spreadsheets, documents, slides, and plain text files (.txt).

Takeouts

The E-Exam system offers a robust solution for securely managing exams in various educational environments. Its plug-and-play design simplifies deployment, requiring no prior software installation. This makes it particularly advantageous for institutions looking to implement a secure and reliable testing environment without significant technical overhead.

The system’s online mode enhances resilience by integrating Moodle for exam delivery and leveraging USB sticks for backup, ensuring data remains safe in the event of technical failures. Similarly, the offline mode provides a straightforward solution for managing exams without internet dependency, maintaining data integrity through periodic backups.

Despite its advantages, the E-Exam system has notable limitations. It relies on specific file types restricting flexibility, excluding other formats that some educational institutions might prefer. In contrast, our project aims to overcome these limitations by minimizing the reliance on students’ personal devices while leveraging external resources to create a more streamlined and resource-efficient solution.

2.1.3 SEB - Safe Exam Browser

Description

SEB [2] is a specialized browser designed to provide a secure and controlled environment for online exams. It transforms a standard computer into a “closed” workstation, limiting access to system functionalities and files, such as internet navigation outside of predefined parameters.

SEB runs on top of the user’s existing operating system, offering the advantage of a straightforward installation process without requiring significant technical expertise. However, this approach also introduces vulnerabilities, as the system’s security primarily relies on blocking applications, leaving potential avenues for bypassing its restrictions. One notable example involves the manipulation or recompilation of its Dynamic Link Library (DLL) files, which can allow the application to run within a virtual machine environment, to run with external monitors, etc — effectively bypassing SEB’s intended limitations and granting the user unrestricted access to both the local system and the internet.

SEB is free and open-source software, making it accessible to a broad audience. While this eliminates costs for educational institutions, it also exposes the system to scrutiny from individuals seeking to exploit potential vulnerabilities.

Takeouts

SEB offers a practical solution for conducting online exams in a controlled environment, particularly for institutions seeking a cost-effective option. Its ease of installation and compatibility with various Learning Management System (LMS), such as Moodle, simplifies integration into existing educational frameworks. By limiting students to a single browser environment, ensures a consistent and uniform testing experience.

However, the reliance on the underlying Operating System (OS) introduces security risks, as advanced users may find ways to bypass its restrictions. Additionally, while the open-source nature of SEB enhances transparency and accessibility, it also allows potential attackers to analyze the software for weaknesses. Finally, its dependency on LMS connectivity and restriction to a single browser may limit the diversity of evaluation methods, potentially constraining examiners who require more flexibility in their assessments.

2.1.4 Exam Control

Description

The Exam Control software, developed by professors at Departamento de Eletrônica, Telecomunicações e Informática (DETI), is designed to create a secure environment for exams while efficiently gathering each student’s exam submissions. This tool automates the process of setting up a temporary, safe account on controlled computers, complete with predefined files in the home directory.

To ensure a secure testing environment, the software configures a customized firewall on each computer and enforces strict permission controls. These measures prevent students from accessing external devices or circumventing restrictions during the exam.

Additionally, the software continuously monitors key parameters on the target computers, including the firewall's status, active user sessions, and changes to files. Exam files are stored locally during the test but can be synchronized with a central server as needed.

Takeouts

The DETI Exam Control system offers a solid foundation for secure exam environments, leveraging robust security mechanisms such as automated account creation, firewall setup, and activity monitoring. By centralizing control and enabling file synchronization with a server, it ensures efficient management and data integrity during examinations.

However, the system's primary limitation is its dependence on controlled university-owned computers, restricting its usability to specific environments. In contrast, our project aims to extend these capabilities by supporting a wider range of devices, including students' personal computers. This expansion will provide greater flexibility for both students and institutions while maintaining the essential security features that make DETI Exam Control effective.

2.1.5 EEL - Eletronic Exam Lab

As it represents the work most closely related to ours, we felt it was important to delve into it in greater detail and provide a more thorough explanation of its features and functionality. This will allow you to better understand this proof-of-concept developed by our predecessors and appreciate its relevance to our project.

Description

Eletronic Exam Lab (EEL)[3] was a project developed with the goal, similarly to our project, of providing a portable environment for academic exams. Developed during the covid-19 pandemic when the need to a good platform for these exams was at its peak.

The student had a controlled system where he would be able to authenticate with Universidade de Aveiro (UA) Identity Provider (IdP) to download the exam (encrypted) but with no write permission. He would then wait for the professor to start the exam to get the write permission so the student can start solving the exam.

The professor had their own platform to upload and control the exams (start, finish, status...) and also control the student, including a live feed for both their screen and their webcam. The professor should also be able to transfer the students' resolutions in an intuitive way.

This project ended being a great proof of concept, as there was never a proper and working implementation for EEL, which we can use for ideas and as base for our project as they still designed a strong architecture.

Takeouts

Figure 2.1 illustrates the architecture of the designed system components, providing a detailed overview of their interconnections and functionalities. We can take a few ideas from here, like using Nginx as a reverse proxy, the API (in Django) directly connected to the database (MySQL). The other components will either not be used or changed as the requirements for our project are significantly different.

We can also overview the communication among the different distributed components in the server and clients. The main server hosts multiple services that orchestrate the communication between the web browsers (professor and student) and the remaining system functionalities. It utilizes technologies such as Python, Nginx, NodeJS and Docker to provide a robust infrastructure.

The client, represented by the student device, contains specific components to manage the network and guarantee the authentication for the user machine. The professor's browser is another point of interaction, also connected to the main server. Through that interface, the professor is able to control the student's environment and visualize the activities and status in real-time. This scheme illustrates a networking system divided into many smaller components, where each one of them has a specific task, always guaranteeing the data safety through asynchronous services and network protocols.

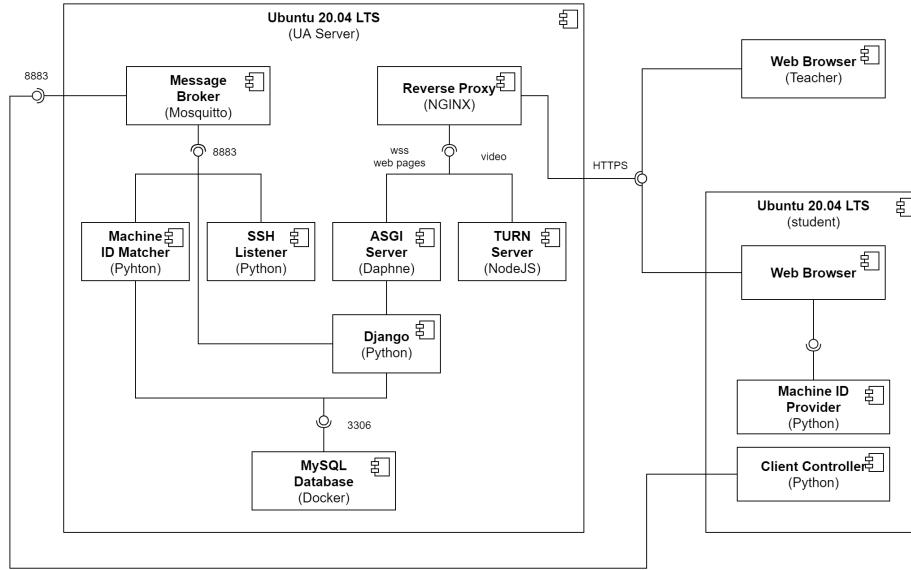


Figure 2.1: EEL Components Diagram.

2.2 TECHNOLOGIES USED

This section provides an overview of the key technologies evolved in the development of the project. It outlines the tools, frameworks, programming languages and platforms that were selected, along with their role in addressing the specific requirements of the project. With that technologies we aim to ensure efficiency and security in our implementation. Additionally, the choices made reflect current best practices and industry standards to achieve a reliable solution.

2.2.1 Isolated System

In our project, the student shall have a dedicated, isolated system where our software is securely stored. The primary objective is to enable seamless integration with their personal computer, allowing them to complete the exam in the simplest and most user-friendly manner

Raspberry PI Zero W

The Raspberry Pi Zero W was the selected hardware due to its compact size, ease of use, and sufficient processing power to support our system without performance issues. Its low power consumption and versatility make it an ideal solution for this project. Additionally, the Raspberry Pi Zero W is an affordable choice that aligns with the project's budget constraints for a large-scale solution .

Flutter

For the software, we have chosen Flutter, a popular open-source UI toolkit developed by Google. With prior experience using Flutter in other projects, we agreed it would be a suitable choice for creating a user-friendly Linux-based interface tailored for student use. Its flexibility and cross-platform capabilities allow us to deliver a seamless and intuitive experience.

2.2.2 Server

Our server will support various functions, divided into three main components: the **Proxy**, the **API**, and the **Web App**. To simplify deployment and maintenance, the API and Web App will be containerized using **Docker**.

API

The API was developed using **FastAPI**, a fast and modern Python framework renowned for its performance, simplicity and comprehensive documentation. This choice ensures a robust backend capable of handling the system's requirements. For data storage, we will use **MySQL**, a reliable and widely-used relational database, to store all necessary information for managing the evaluation process.

Webapp

The Web App will serve as the central hub where professors and administrators manage the system. To develop this component, we have chosen a combination of **Node.js** and **Next.js** for their complementary strengths and modern capabilities.

Node.js will handle the backend aspects of the application, providing a flexible and high-performance runtime environment for server-side logic, API handling, and data management.

Next.js, a React-based framework, will power the frontend of the application. Its ability to support server-side rendering and static site generation enhances performance and improves search optimization. This software also simplifies routing and state management, making it easier to deliver a user-friendly and polished interface for professors and administrators.

By combining these two technologies with prior experience using them by our team, we can create a cohesive, modern, and scalable Web App that provides an intuitive experience for users while maintaining the technical efficiency needed for seamless system management.

Proxy

To enable seamless communication between components and ensure efficient data flow, we will implement a proxy server using **Nginx**. This powerful, free, open-source software is ideal for managing connections, load balancing, and ensuring secure and reliable communication between the server's elements.

Conceptual Model and Architecture

3.1 CONCEPTUAL MODEL

The system must respond to certain functionalities so that the user can achieve their objectives. Therefore, below are the following tables with the main functional and non-functional requirements. They were divided into several parts, such as the webapp, the isolated system and the server.

3.1.1 Functional Requirements

The functional requirements we identify are the result of a thorough analysis of the project goals. These requirements outline the specific functionalities that must be implemented to ensure the success of the project.

Webapp

The web application will primarily be used by professors to create and monitor exams, providing a user-friendly and effective way to create and coordinate the exam process. The admin will use this interface to perform maintenance tasks and ensure an easier way to configure multiple devices at the same time. The Table 3.1 outlines the specific requirements identified for the Web Application.

Table 3.1: Functional Requirements - WebApp.

Reference	Functional Requirements
FR-1	The system should be able to upload the exams to the server and download the resolutions to grade them;
FR-2	Should be able to monitor students' progress and activity in real-time so that the user can ensure they're complying to the exam rules and detect any issues.;
FR-3	To know when a student has a connection issue so that the user can provide support and help them resolve it, being able to pause or end a student's exam session if necessary so that he can ensure exam integrity in cases of suspected rule violations;
FR-4	Identify each student before they access the exam. This ensures that only authorized students take the exam.;
FR-5	Define the start time and duration of the exam so that all students follow the same schedule and time limit.;
FR-6	The system must be able to monitor the system's performance during exam periods so that we can identify and solve issues;
FR-7	To know when all students have submitted their exams so that the user can confirm the exam session has ended. If necessary, he can do it manually so that he can verify that the exam is officially concluded and students can no longer access it;
FR-8	Want to be able to perform regular maintenance without disturbing exam sessions;

Isolated System

The device utilized by students to access an Isolated System must facilitate their navigation within a familiar and uncomplicated environment, while concurrently ensuring the integrity and security of their personal laptop. The device should also provide an integrated storage solution to ensure the backup and preservation of all files, thus mitigating the risk of data loss. The Table 3.2 lists the identified requirements that the isolated system must meet.

Table 3.2: Functional Requirements - Isolated System.

Reference	Functional Requirements
FR-9	The student must not have access to anything that is not strictly necessary to carry out the exam proposed by the professor;
FR-10	The system interface must allow all students to feel comfortable taking the tests presented to them, using their preferred peripherals;
FR-11	The student should start the exam only when the professor starts it;
FR-12	The student should be able to authenticate to the system;
FR-13	The student should be able to enter the system, read his exam and start doing it;
FR-14	The system should make backups every certain amount of time to prevent losing data;
FR-15	Allow the student to know how much time he has left to take the exam, so he can manage his time;
FR-16	Allow the student to mark the question to revisit it easily;
FR-17	Ensure that personal files or data will not be accessed by the exam environment thereby preventing any malware or harmful software so that the student can trust connecting his laptop without any risk;
FR-18	Being able to disconnect easily if an error occurs without damaging the personal device;
FR-19	Allow physical access to the configuration mode;

Server

The main server requirements identified are shown in the Table 3.3, this requirements are critical to ensure secure data storage and communication and handle the authentication processes.

Table 3.3: Functional Requirements - Server.

Reference	Functional Requirements
FR-20	Should block every connection with the exterior;
FR-21	Should be able to transfer the data between the isolated system and the webapp;
FR-22	Should authorize access for the students who are signed up for the test;
FR-23	Be able to ensure all data in the server is securely stored and regularly backed up so that no data is lost in case of a technical failure;

3.1.2 Non-Functional Requirements

The non-functional requirements on the Table 3.4 are based on a thorough evaluation of the system's operational context and performance expectations. They complement the functional requirements by focusing on how the system performs its tasks, rather than what tasks it performs.

Table 3.4: Non-Functional Requirements.

Reference	Non-Functional Requirements
NFR-1	Usage of UA idP for the authentication;
NFR-2	All communications must be secure and data must be stored securely;
NFR-3	The system must tolerate network and power failures;
NFR-4	The students must interact with the system without having knowledge of the backend;
NFR-5	No problems with lagging or lack of performance;
NFR-6	Ensure compliance with GDPR(General Data Protection Regulation) so that we protect all information and avoid legal issues;
NFR-7	The system should let the user create their exams in an intuitive and easy way using specific tools and resources;

3.1.3 Actors

This section describes the main actors involved in the project, detailing their roles, responsibilities and impact on the overall objectives. The role of each actor is analysed to understand how their contributions and interactions influence the development of the project, ensuring a coordinated approach to achieving the desired results.

After this brief analysis of the actors, the main actors of the system were defined, including professors and students. Given the objectives to be achieved, it would be important to mention the following aspects about the main actors:

- **Professor:** Their objective is to optimise the evaluation methods of the Curricular Units (CU) he teaches. Reduce the time spent creating tests and exams, and minimise the possibility of any type of fraud attempt during exams. Professors will have the possibility to keep a close watch on all your students who are taking the exam at that particular time.
- **Student:** Their role is to test the secure environment. It is this entity that will be subjected to testing with the aim of not being able to cause fraud. When carrying out the assessment moments in this new environment, students will have the possibility of carrying out an examination on the personal computer in a convenient way.
- **Admin:** The Admin is the central entity responsible for ensuring that the examination system operates in a stable and secure manner, especially during critical evaluation periods when traffic peaks. Their role is to constantly monitor the system performance, ensuring it maintains optimal functionality even under heavy load. One of the Admin's primary responsibilities is managing the system infrastructure. This includes configuring servers, implementing load balancing, and ensuring redundancy to avoid single points of failure. Additionally, the Admin is responsible for protecting sensitive data, such as student and exam information. The Admin is the backbone that keeps the system running smoothly and securely, ensuring that all users can carry out their activities without interruptions or risks. Their mission is to ensure the platform's reliability, security, and success, allowing users to focus on academic tasks without technical concerns.
- **Auditor:** The Auditor is a key entity responsible for ensuring that the system meets all security requirements and the privacy rights of its users. Their role involves continuous remote monitoring of all activity within the system, providing a comprehensive view of its operation. The Auditor is tasked with analyzing data and user behaviours to identify potential anomalies, such as attempts at fraud or information theft. Their mission is to uphold the trust of all users by ensuring that their data remains protected and that the system operates transparently and securely, free from malicious interference.

3.1.4 Uses Cases

This section will present the possible use cases of each actor in our system and the possible interactions with it. As seen, we will have four main entities using our system, the professor, the student, the Admin and the Auditor.

WebApp Environment

This use case diagram represents the interaction between a Professor and a WebApp system. The Professor interacts with the WebApp primarily to manage the lifecycle of exams, starting from their creation to their completion and review.

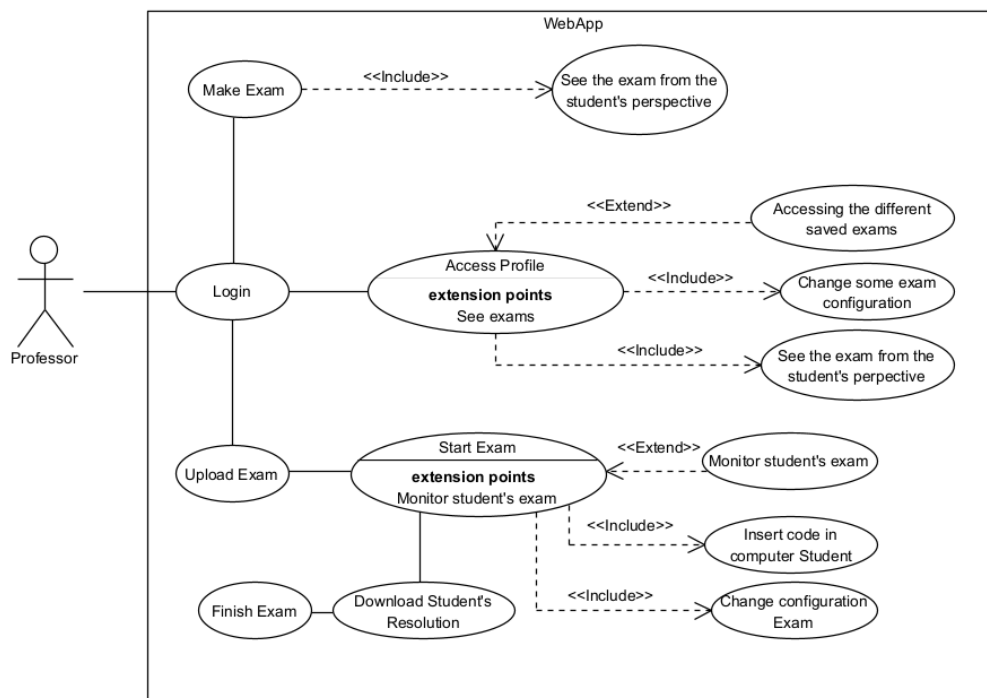


Figure 3.1: Use Case Diagram for Professor.

Now each of the Professor's interactions with the use cases will be explained:

Login:

- This is the first step. Involves logging into the system through the **Login** use case, which grants access to the professor's profile. Once logged in, the Professor can interact with several functionalities provided by the WebApp.

Access Profile:

- After logging in, the teacher can access your profile. In the profile you can perform the following operations:
 - **Accessing the different saved exams** → A professor may have associated several CU's and in addition several exams for each of these CU's. With this it would be useful to be able to view the different exams that you have in the profile.
 - **Change some exam configuration** → After taking an exam, a teacher can make changes to a specific exam without having to create a new exam.
 - **See the exam from the student's perspective;**

Make Exam:

- The Professor can create new exams through the **Make Exam** use Case. When creating the exam, the WebApp includes an option:
 - **See the exam from the student's perspective** → This interaction ensures that the exam meets the desired format and clarity standards before it is uploaded.

Upload:

- After preparing the exam, the Professor can use the **Upload Exam** functionality to add the exam to the system. After uploading, then the teacher can **Start Exam**. During the exam, the professor can:
 - **Monitor student's exam;**
 - **Insert code in computer student;**
 - **Change configuration Exam;**
 - **Download Student's Resolution;**
- Once the exam is completed, the Professor uses the **Finish Exam** functionality to conclude the process.

Secure Environment

In Figure 3.2 we can see the Use Cases for secure environment and the interactions between the main actor (**Student**) and the **Secure Environment system**.

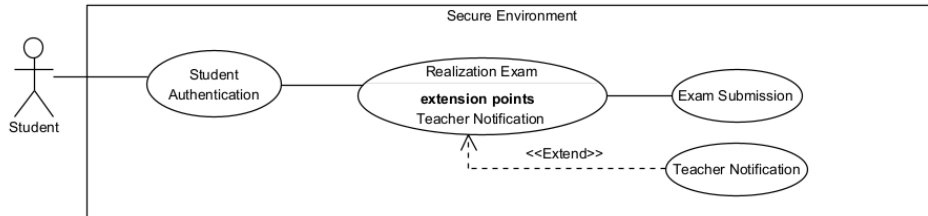


Figure 3.2: Use Case Diagram for Student.

With this, we can enumerate the following main use cases:

Student Authentication:

- The process begins with **Student Authentication** use case. This step ensures the student's identity is verified before accessing the exam. This is a mandatory use case for security and access control in the secure environment.

Realization Exam:

- After successful authentication and authorization from professor, the student proceeds to take the exam through the Realization Exam use case.
 - **Professor Notification** → During the exam, certain actions or conditions may trigger notifications to the student from the professor. This notification can be used to show the student the remaining time, refer about some doubts or a rule violation.

Exam Submission:

- After completing the exam, the student submits their work via the **Exam Submission** use case. This final interaction concludes the student's participation and securely stores the exam data for evaluation.

Use Case Diagram for Admin

The Admin interacts with the system to perform management and monitoring tasks, as we can see in Figure 3.3.

These interactions emphasize the Admin's central role in supervising and maintaining the system, ensuring that the environment remains secure, functional, and well-organized.

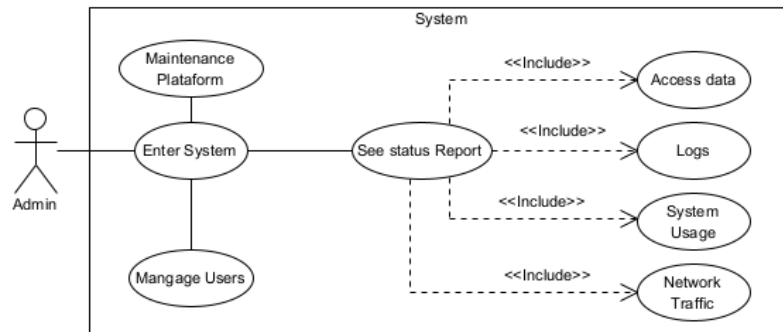


Figure 3.3: Use Case Diagram for Admin.

In this case, we can enumerate the following main use cases:

Enter System:

- This is the primary entry point for the Admin. From here, the Admin can either access the Maintenance Platform for technical management or Manage Users to handle system users.

Maintenance Platform:

- This feature provides the Admin with tools specifically designed for system maintenance.

Manage Users:

- The Admin can manage user accounts within the system, which includes creating, modifying, or deleting accounts and assigning appropriate access permissions to ensure a secure and efficient user environment.

See Status Report:

- This is a vital feature that allows the Admin to access detailed system reports.
 - The See Status Report use case encompasses several other essential functionalities, as indicated by the «include» relationships. These are:
 - **Access Data:** This provides a comprehensive view of system access information, including login records, timestamps, and failed login attempts.
 - **Logs:** The Admin can review detailed activity logs, which record all actions performed within the system.
 - **System Usage:** This feature offers insights into the system's resource utilization, such as CPU usage, memory consumption, and storage capacity.
 - **Network Traffic:** This allows the Admin to monitor the system's network traffic.

Use Case Diagram for Auditor

The use case diagram illustrates the interactions between the Auditor and the system, focusing on ensuring security and transparency. The Auditor logs into the system to access various functionalities, such as viewing activity logs (action and access logs) and monitoring exams for fraud attempts. If anomalies are detected, the system can notify the Auditor. Additionally, the Auditor can access system configurations and report suspicious configurations through anomaly detection, reinforcing the system's integrity and trustworthiness.

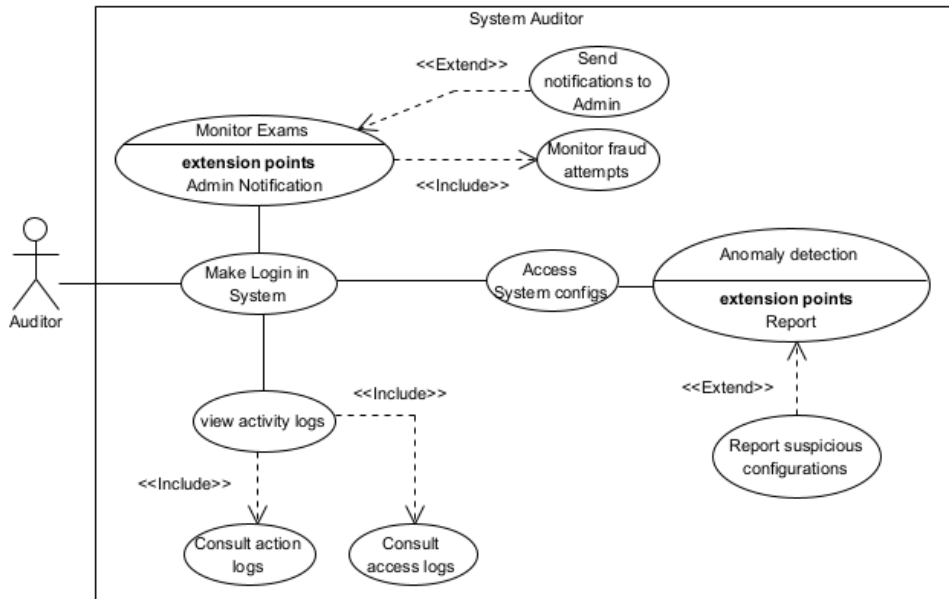


Figure 3.4: Use Case Diagram for Auditor.

The Auditor, as the main actor, interacts with several use cases:

Make Login in System:

- The entry point for the Auditor, where authentication is performed. Once logged in, the Auditor gains access to the system's functionalities.

Consult Action Logs:

- This use case allows the Auditor to analyze system operations. It includes two detailed subcases:
 - **Consult Action Logs:** Access logs detailing user actions within the system.
 - **Consult Access Logs:** Review logs related to system access events.

Monitor Exams:

- The Auditor oversees exam-related activities. This process includes monitoring for potential **fraud attempts**, and if anomalies are detected, an extension point enables the system to **send notifications to the Auditor**. This ensures rapid action in response to suspicious activity.

Access System Configuration:

- The Auditor can view and manage system settings to maintain optimal operation and security.

Anomaly Detection:

- This is a proactive use case where the system automatically scans for unusual behaviors or configurations. It includes an **extension point**, allowing the Auditor to **report suspicious configurations**, reinforcing security oversight.

3.1.5 Abuse Case

This project is designed to be used in different ways depending on the user. The project's architecture (seen in Figure 3.5) is designed in a way to separate those roles so we only have to worry about this issue in one module without risking the overall system's integrity.

Student:

- **Submission of plagiarized or automatically generated assignments:**
 - Students may attempt to submit work that is copied from other sources or generated using automated tools to bypass genuine effort and earn credit dishonestly.
 - **Relevance Level (High):** Submission of plagiarized or automatically generated assignments, as this directly undermines the purpose of the educational system.
- **Exploitation of system vulnerabilities to gain unfair advantages:**
 - Students might try to exploit technical flaws or weaknesses in the system, such as altering grades, accessing unauthorized content, or manipulating exam remaining time, thereby undermining the system's integrity.
 - **Relevance Level (High):** Exploitation of system vulnerabilities, since it can compromise sensitive data and the integrity of the entire academic process.

Administrator:

- **Alteration of permissions or deletion of sensitive data:**
 - Administrators may misuse their elevated privileges to modify user permissions or delete critical data, potentially favoring specific individuals or groups.
 - **Relevance Level (Medium):** Alteration of permissions or deletion of sensitive data, as it can have a significant impact on system operations and fairness.
- **Creation of fictitious accounts to manipulate reports:**
 - Administrators might generate fake accounts to inflate system usage statistics or bypass accountability mechanisms, compromising the integrity of system reports.
 - **Relevance Level (Low):** Creation of fictitious accounts, as it primarily affects data reporting and system analytics.

Professor:

- **Exploitation of direct configuration mode access to install unauthorized software:**
 - Professors may attempt to exploit the access to the system’s configuration mode to install unapproved software or alter the device settings before an exam, potentially harming the student device or disrupting the examination process.
 - **Relevance Level (High):** This abuse is highly relevant due to the significant impact on the integrity of the examination process and the fairness of the academic environment.
- **Manipulation of data to obscure performance or inefficiencies:**
 - Professors might alter system data, such as attendance records or grade distributions, to conceal their own shortcomings or teaching inefficiencies, compromising the system’s transparency.
 - **Relevance Level (Low):** Manipulation of data to obscure performance, as it has limited immediate effects but erodes the system’s credibility.
- **Unauthorized retrieval of confidential student information:**
 - Professors may exploit their access to the system to extract sensitive data from students’ devices or accounts, violating privacy and ethical boundaries.
 - **Relevance Level (High):** Unauthorized retrieval of student information, due to its significant ethical, legal, and reputational implications.

3.2 ARCHITECTURE

This section presents an overview of the system architecture, describing its domain model, the multiple components and the respective relationships, its physical and technological model. In this section we can see the sequence of events during the student's interaction with the environment, the professor's interaction with the system, and the exam process.

3.2.1 Components

The system is composed of three primary modules: the Application Programming Interface (API), the secure environment and the WebApp. Each of these three modules play a crucial role in the execution flow of the system's functionalities, as we can see in Figure 3.5.

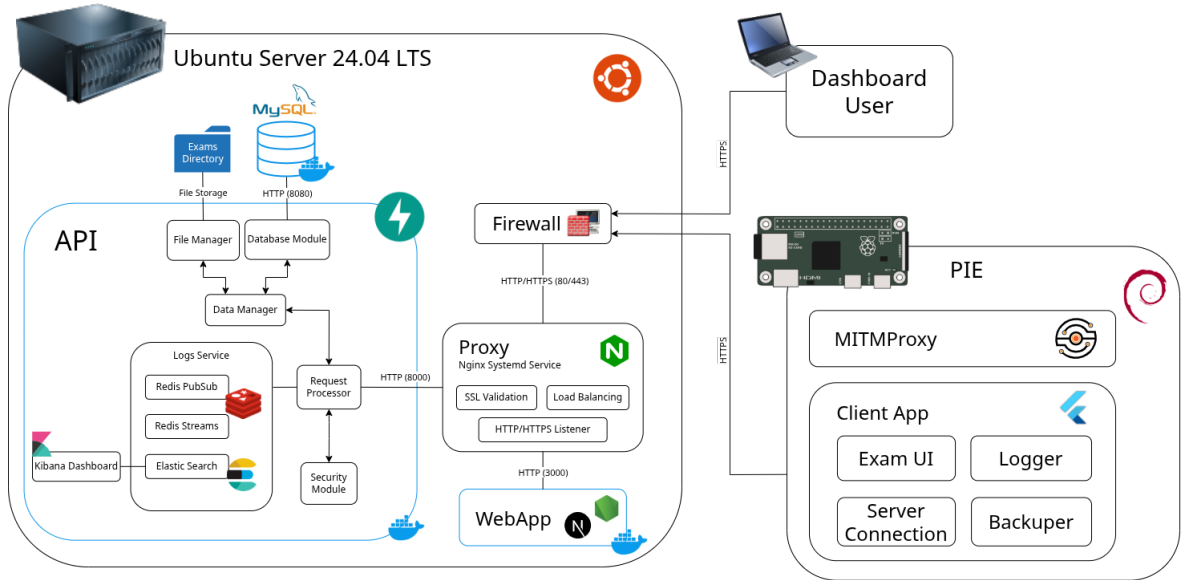


Figure 3.5: Architecture Diagram.

Server

This is where both the API (FastAPI) and WebApp (NextJS) are hosted. The API handles and provides all the necessary services to the WebApp and the PIE (Debian). It also serves the message broker (Redis PubSub) used to simultaneously control all the raspberries, as well as serving as a bridge to the database (MySQL). Finally, it provides the admin with the system info dashboard (Kibana) where he can see the logs and other information. These services are behind a Nginx proxy responsible for the Secure Sockets Layer (SSL) validations, load balancing and manages Hypertext Transfer Protocol (HTTP)/Hypertext Transfer Protocol Secure (HTTPS) traffic (enforcing HTTPS).

API Components:

- **File Manager:** Responsible for managing file storage. It manipulates the server's file system, ensuring that data is stored, organized, and retrieved both efficiently and securely as all the stored files are encrypted with keys not stored on the server.
- **Database Module:** Direct communication with a relational database, in this case MySQL. This module is used to store critical information such as user credentials, system states, and other persistent data.
- **Data Manager:** It acts as an orchestrator that connects the File Manager and Database Module. Processes requests, validates data, and provides standardized responses to customers.
- **Request Processor:** It processes incoming HTTP requests, validating headers, authenticating JWT tokens (if used), and directing requests to the appropriate modules.
- **Security Module:** Implements security measures such as:
 - Encryption/hashing of confidential data;
 - Prevention against brute force attacks and SQL injection;
 - Secure session management.
- **Logs Service:** Implements the logs receiver/generator and provides the respective visualisation with the following components:
 - **Redis PubSub:** The logs are sent as messages to the redis broker;
 - **Redis Streams:** Provides a quicker access for the most recent logs;
 - **Elastic Search:** Provides a more efficient way of storing and retrieving logs;
 - **Kibana:** Provides the visual dashboard with the information displayed correctly.

Proxy:

- **SSL Validation:** Ensures secure communications through TLS certificates.
- **Load Balancing:** Distributes requests between multiple server nodes, optimizing performance and availability.
- **HTTP/HTTPS Listener:** Monitors standard ports (80/443) for incoming connections, redirecting requests to the appropriate module. All the HTTP requests are rewritten into a HTTPS request.

Portable Isolated Environment

This is the environment the student uses during the exams. It's a Debian 12 based system with just the necessary features to provide the user with an automated exam process and the tools to solve the exam.

It contains the following two main components:

- **MITMProxy:** Program that filters the accessible pages so that the user can only access the pages allowed by the professor.
- **Client App:** Built in flutter, provides the following components:

- **Exam UI:** Graphical interface that allows the user to perform the necessary tasks such as logging into the session, seeing notifications and allowed URLs. It performs some automated tasks such as downloading the exam and submitting the solution. It also contains the user in some specific situations not allowing them to use the system before/after the session is underway;
- **Logger:** Monitors and publishes to the server changes in the system, such as an initial system information log, the USB devices being connected/disconnected, executed commands, etc.;
- **Server Connection:** Module responsible for establishing the connection to the server, listening and publishing messages with the broker and ensuring the connection is not lost;
- **Backuper:** Ensures the files are saved in the case of a system failure.

3.2.2 Workflows

Activity Diagram - Exam Creation

The first one we will look into is the exam creation diagram and it shows the general workflow that professors follow when doing so.

It starts simply with the professor accessing the platform and authenticating to the platform using UA IdP. From there, the professor will have the list of their courses and may select one to create the exam. Making the exam can be either making a form-like test or uploading a set of files and setting some rules for the student to solve in the PC instead of a straight up list of questions. Upon finishing making the exam, there's the option to preview it, either opening a web page in the case of the form-like test or opening a virtual environment to exemplify the student desktop in the case of having a set of files to solve.

This diagram may be seen in Figure 3.6

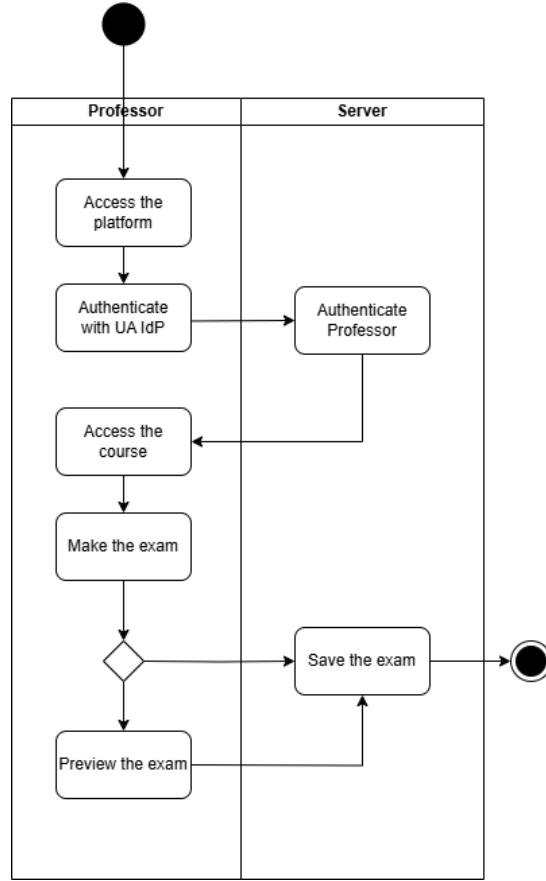


Figure 3.6: Activity Diagram - Exam Creation.

Activity Diagram - Exam Solving

The exam solving flow starts with the student connecting the device to their personal laptop, after changing the boot order, the student successfully boots from the device, thereby creating the isolated environment. On startup, the device sends a connection request to the server. The server checks the device's integrity, authorizing it if so. If this process fails the server rejects the connection and notifies the student of the failure. The next step would be for the student to log in using UA IdP to associate the device with that student. Upon successful login, the student must connect to an exam session, thereby associating the student with that session. The user then receives the exam from the server and the device waits for the professor's command to allow the exam to start. Once all students have joined the session and the professor confirmed all the information, he starts the exam. While the student solves the exam, the device periodically backs up the exam so that nothing is lost in the event of an error and an exam interface is displayed so that the student can track the time remaining and submit the exam when finished. At the same time, the professor monitors all the students and when all the students have finished the exam, or when the professor decides, he can end the exam session. At the end of the session the professor requests the solved exams from the server and downloads them for grading. This diagram may be seen in Figure 3.7

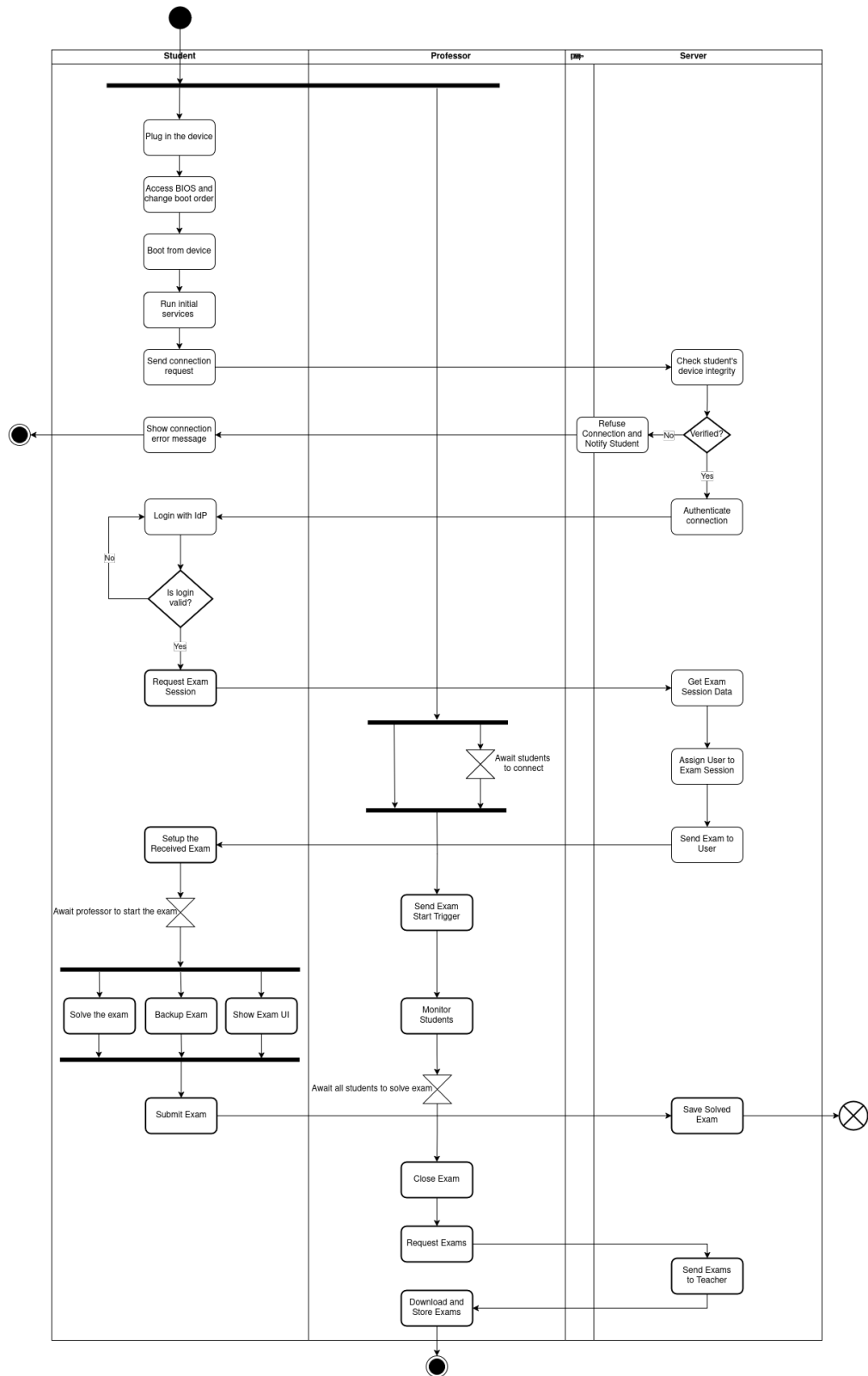


Figure 3.7: Activity Diagram - Exam Solving.

3.2.3 Interaction Diagrams

The Interaction Diagrams shown represent the information and actions exchanged within the system. The processes we dive into are related to the evaluation process and the device configuration process, the latter of which may be carried out in two distinct ways.

Interaction Diagram - Evaluation Process

The Interaction Diagram in Figure 3.8 illustrates the different interactions that take place in the system (between student, professor, server and the device itself) for the evaluation process.

The Professor connects to the server using his institution's credentials to log into a web application. He then sends an exam to the server to be available for students to solve it. The Professor also receives the student's information when the student enters the exam session. Once everything is set up, the professor can start and stop the exam as needed. When the exam is finished the professor receives the solved exams from the server.

The Student plugs in the device to create the environment, which sends its information to the Server to confirm its authenticity. After this validation, the Device asks for the Student's credentials. When the Student responds with their credentials, the Server associates the student with the device they are using. The Device again asks for a session code, and when the Student responds, the Device is associated with an exam session from which the Server sends the encrypted exam. When the exam starts the Device receives the decryption key and can then display the decrypted exam to the Student, who then sends the solved exam through the Device, which encrypts it before sending it to the Server.

At one end, the Device must handle student authentication, session association and exam delivery. At the other end, the Server must verify credentials, manage session associations, ensure secure transfer of exams and handle exam storage. This flow ensures a secure and reliable process to maintain exam session integrity and reliability.

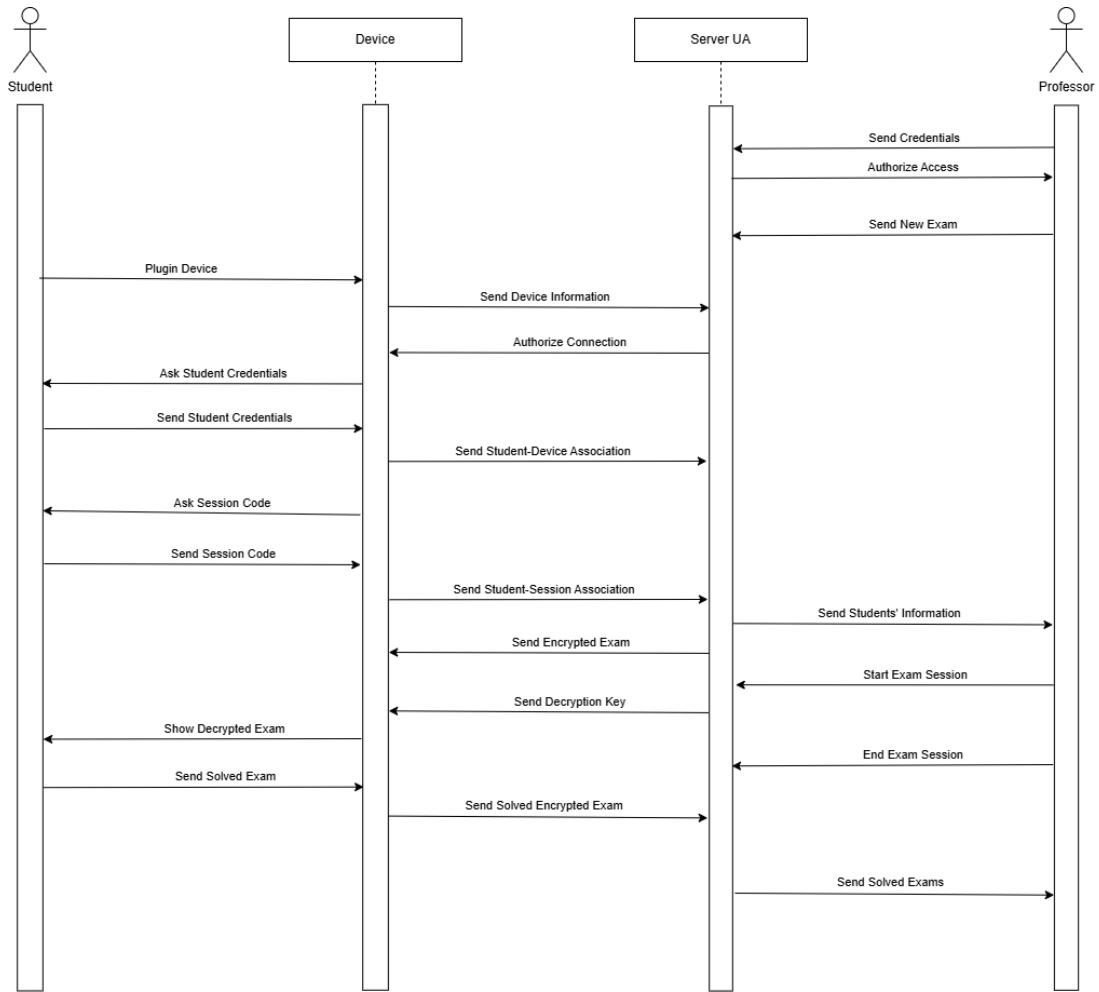


Figure 3.8: Interaction Diagram - Evaluation Process.

Interaction Diagram - Direct Device Configuration

Let us now look at the process of configuring the Device, which, as mentioned earlier, can be done in two different ways. The figure 3.9 illustrates the process for directly configuring a Device through physical interaction, detailing the interaction between the Admin, the Device and the Server.

The process begins with the Admin booting the Device into configuration mode. On startup the device asks for admin credentials, after the Admin provides their credentials the Device verifies them with the Server to authorise access to configuration mode. Upon successful authorisation, the device immediately sends a backup of its current state to the Server to ensure a record of its pre-update configuration.

The Admin can now initiate the configuration and maintenance tasks, such as system and security updates. During this process the Device provides feedback to the Admin, such as update progress and completion status. As part of this process, the Device also makes its system information available to the Admin for review. Finally, a detailed maintenance log, capturing all actions performed during this session, is sent to the Server for auditing and tracking purposes

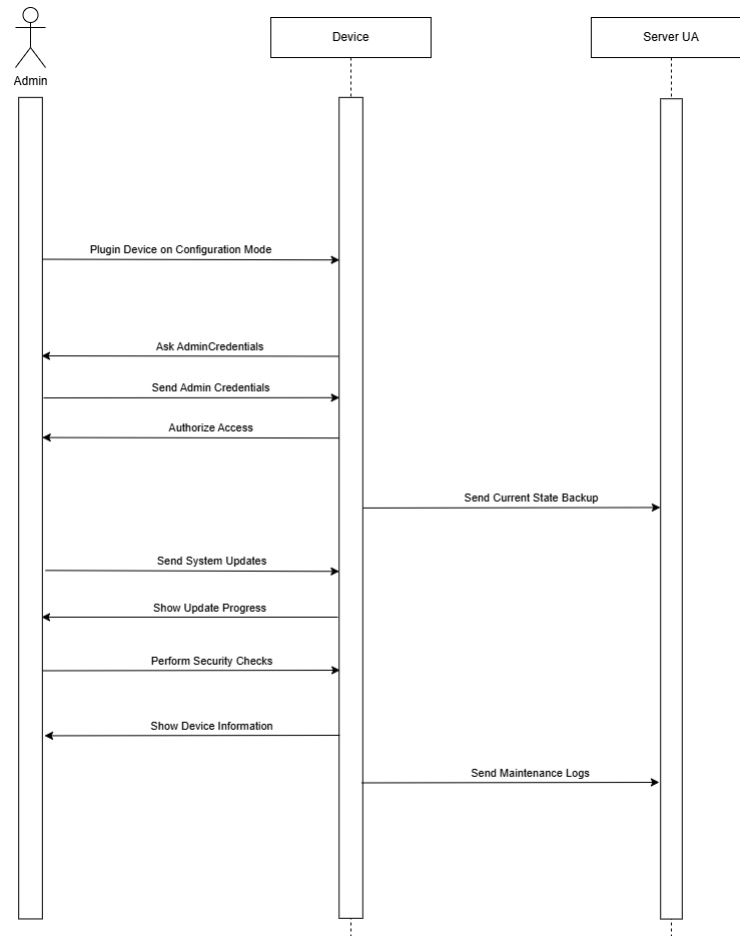


Figure 3.9: Interaction Diagram - Direct Device Configuration.

Interaction Diagram - Remote Configuration

We now take a look at the remote configuration option as depicted in figure 3.10. This method allows the Admin to manage multiple devices simultaneously through a centralized interface.

The process begins with the Admin accessing the Server's Maintenance Platform by providing their credentials. The Server verifies these credentials to authorise access to the Maintenance Platform. Once access is granted, the Admin is presented with a list of connected Devices and their statuses, retrieved from the Server after querying the Devices. The Admin selects the Devices they wish to configure or update. After the selection, the Server sends commands to the chosen Devices to switch them into configuration mode. Each Device responds by sending a backup of its current state to the Server to ensure a recovery point is available if needed. Once the backups are confirmed, the Admin initiates the configuration and maintenance tasks on the selected Devices. Throughout the update process, the Server provides real-time progress feedback to the Admin, ensuring visibility into the status of each Device. Finally, the Devices send detailed information back to the Server, including logs of the updates performed, their current state, and the results of the security checks. These logs are stored centrally for auditing and troubleshooting purposes.

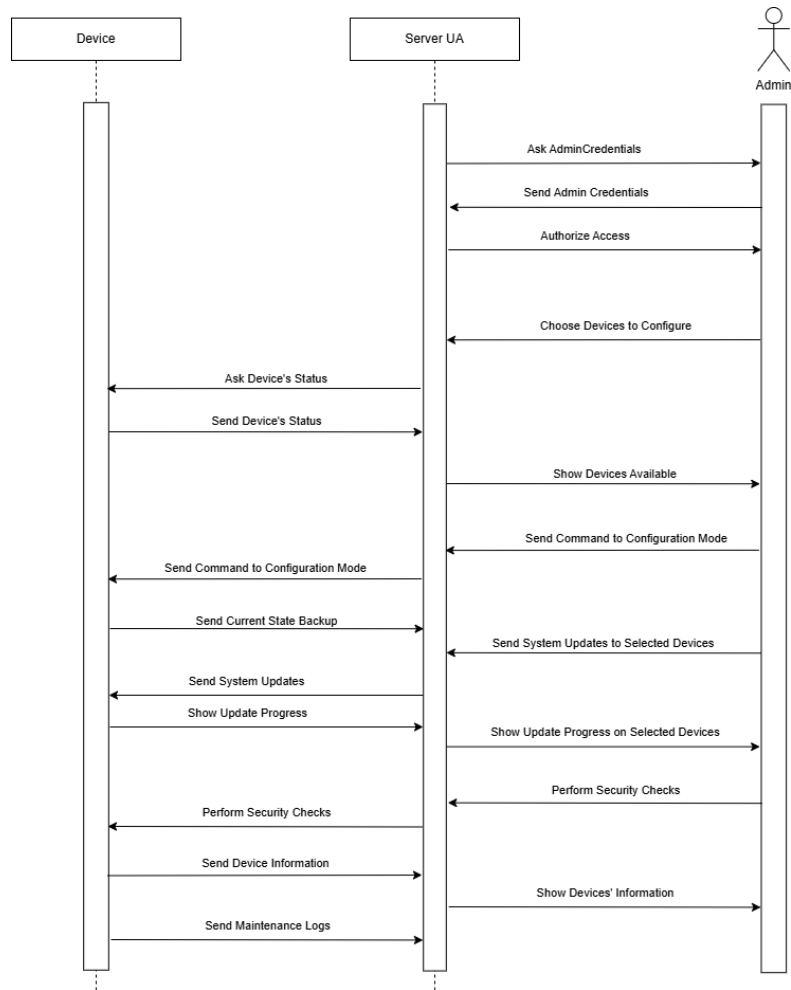


Figure 3.10: Interaction Diagram - Remote Configuration.

Implementation

4.1 SERVER

This section provides an overview of the server implementation, detailing the architecture, running services, and their configurations. The server implementation is built around a modular architecture, composed of dedicated components that work together to provide a robust and cohesive backend solution. The core system consists of the main API, database layer, and logging services, with additional supporting modules that can be easily modified or replaced. This modular approach allows for better maintainability, scalability, and separation of responsibilities across all services.

This modularization is implemented using Docker containerization, which ensures consistent deployment across different environments and simplifies service management. The complete Docker Compose configuration is attached in Appendix A.4

4.1.1 API

The API was developed using FastAPI [4], a modern, Python-based web framework for building APIs. FastAPI is known for its high performance, ease of use, and automatic generation of OpenAPI documentation. We chose FastAPI due to our familiarity with the Python ecosystem and the ease of integrating it with other components of the system. The API serves as the main entry point for all client requests, handling authentication, authorisation, and routing to the appropriate services. It is designed to be stateless, allowing for horizontal scaling and load balancing.

Router Structure

The API was designed to follow a modular structure with separate routers for each functional domain.

- **Course Router:** Handles course management options;
- **Exam Router:** Manages exam operations within specific courses;
- **Session Router:** Controls session lifecycle within exams;
- **User Router:** Manages user authentication and management;
- **Device Router:** Handles hardware device registration and monitoring;
- **Session-User Router:** Manages student participation in sessions;
- **Submission Router:** Handles file upload and submission management;
- **PIE Router:** Responsible for real-time communication between the device and the system;
- **Logs Router:** Manages system logging;
- **Screenshare Router:** Enables screen-sharing functionality;
- **Analytics Router:** Provides data analytics and reporting endpoints.

The endpoints are organised in a hierarchical structure that reflects the defined workflow as seen in A.3

This structure ensures that all session operations inherently belong to a specific exam within a specific course, maintaining proper data isolation and access control.

Authentication and Authorization

The API implements JWT-based authentication with role-based access control, supporting student, professor and admin roles. Detailed security implementation is discussed in Section 5.1.

Real-time Communication

Beyond REST endpoints, the API implements WebSocket connections for real-time screen sharing functionality. This enables bidirectional communication necessary for interactive exam monitoring and control, allowing instructors to view student screens during exam sessions.

The screenshare system is built on a hybrid architecture combining WebSocket connections for signaling and Web Real-Time Communication (WebRTC) [5] for direct peer-to-peer media streaming, with Redis Pub/Sub facilitating message exchange between participants.

The screen sharing process follows the standard WebRTC signaling flow:

1. **Connection Establishment:** Student connects to WebSocket endpoint and is registered as available
2. **Screen Share Request:** Monitor requests screen sharing from a specific student via WebSocket message

3. **Offer Creation:** Student captures screen media and creates WebRTC offer with optimized constraints
4. **SDP Exchange:** Offer/Answer exchange is facilitated through Redis Pub/Sub channels
5. **ICE Candidate Exchange:** Network connectivity information is exchanged for NAT traversal
6. **Direct Connection:** WebRTC establishes direct peer-to-peer connection for media streaming

Through this approach, we aim for scalability and performance since, despite the initial exchange of information being made through the server, the communication continues directly between professor and student without overloading the server.

Error Handling and Validation

The API implements comprehensive error handling with structured HTTP status codes and detailed error messages. Request validation is performed using Pydantic models, ensuring data integrity and providing clear feedback for invalid requests.

API Documentation

FastAPI automatically generates comprehensive OpenAPI documentation, accessible through interactive Swagger User Interface (UI) and ReDoc interfaces. This self-documenting approach ensures that API endpoints, request/response schemas, and authentication requirements are always up-to-date and easily accessible to developers and integrators.

4.1.2 Database

The database is implemented using MySQL, a widely used relational database management system. MySQL was chosen for its robustness, scalability, and strong community support. The database schema is designed to support the modular architecture of the API, with separate tables for each functional domain.

To handle database interactions we use SQLAlchemy, as the Object-Relational Mapping (ORM) layer, which provides a high-level abstraction for database operations and simplifies the management of database connections and transactions.

The database schema is represented using SQLAlchemy models, which are mapped directly to MySQL tables, as illustrated in the Entity-Relationship Diagram (ERD) in 4.1

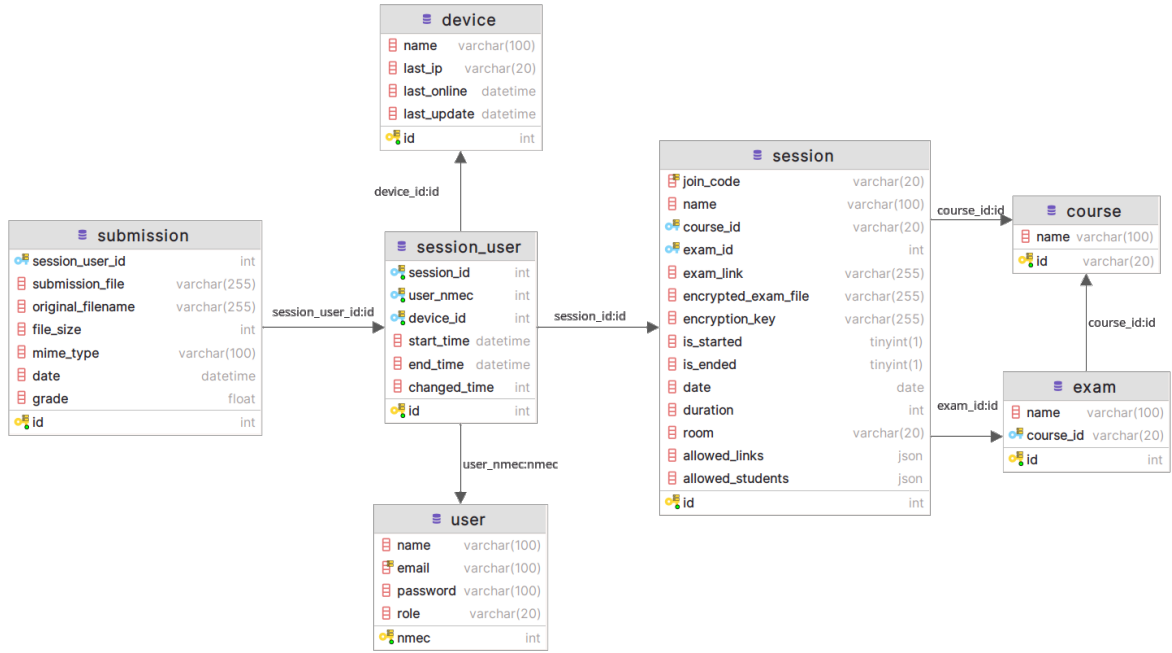


Figure 4.1: Database ERD.

4.1.3 File Manager

The File Manager is responsible for handling all file operations within the system, providing a centralized approach to file storage, organization, and retrieval. It operates through the API endpoints but maintains its own dedicated logic for file handling operations.

Files undergo validation processes to ensure they meet system requirements before being permanently stored. This processes will be later discussed in 5.3. The system maintains metadata about each file including original filename, file size, MIME type, and upload timestamp.

Storage Architecture

Files are organized using a structured directory hierarchy that reflects the database relationships. The system automatically creates paths based on course names, exam identifiers, and session information, ensuring consistent organization.

For exam materials, the system supports both file-based and link-based content delivery. When multiple files are uploaded for a single session, they are automatically packaged into a compressed archive for efficient distribution.

Student submissions are stored in dedicated directories with timestamped filenames to prevent conflicts and maintain submission history.

4.1.4 Logs Service

The Logs Service is responsible for capturing, storing, and visualizing system logs. It provides a comprehensive view of system operations, errors, and user activities, enabling effective monitoring and auditing.

It is also through this service that the system can provide real-time updates to the PIE, allowing for device control and monitoring.

Redis PubSub

The Redis Pub/Sub service is used to handle real-time messaging between the server and connected devices. It allows the server to send messages to the PIE, enabling real-time updates and control commands. The PubSub mechanism ensures that messages are delivered immediately to all subscribed clients, facilitating efficient communication.

The choice of Redis Pub/Sub was motivated by its simplicity, low latency, and ability to efficiently handle real-time communication. Redis Pub/Sub provides a lightweight publish/subscribe messaging pattern that is well-suited for scenarios where messages need to be broadcast instantly to multiple subscribers, such as sending control commands or status updates to all connected PIE devices.

Additionally, Redis is already used in the system for other purposes, which reduces operational overhead and simplifies deployment. Its in-memory architecture ensures minimal delay in message delivery, making it ideal for time-sensitive operations required by the system.

The Webapp 4.2 interacts with this service to send messages in real-time to devices and also session control.

Redis Streams

To ensure efficient access to the most recent logs and facilitate advanced log processing, Redis Streams is integrated with Pub/Sub. Redis Streams is a data structure designed for the storage and retrieval of log messages in an append-only, time-ordered sequence. Unlike Pub/Sub, which delivers messages only to active subscribers, Redis Streams retains messages, allowing consumers to read logs at their convenience and replay past entries when necessary.

By utilizing Redis Streams, the system guarantees the persistence of log messages, even if consumers are temporarily offline or need to process logs asynchronously. This combination of Redis Pub/Sub for real-time message delivery and Redis Streams for durable, ordered log storage provides a comprehensive and adaptable logging infrastructure that satisfies both immediate and long-term monitoring needs.

ElasticSearch

Elasticsearch serves as the long-term storage and search engine for system logs, providing advanced querying, aggregation, and analysis capabilities. The implementation utilizes the AsyncElasticsearch client to ensure non-blocking operations and optimal performance within the FastAPI asynchronous framework.

The implementation provides comprehensive search functionality supporting:

- **Multi-field filtering:** Simultaneous filtering by device, user, session, course, exam, level, and category
- **Time range queries:** Efficient filtering of logs within specific time periods
- **Pagination support:** Scalable result retrieval with configurable page sizes
- **Sorted results:** Chronologically ordered logs with newest entries first

This implementation provides the foundation for comprehensive log analysis, academic monitoring, and system analytics while maintaining high performance and reliability standards.

Kibana

Kibana serves as the primary visualization and dashboard platform for the logging infrastructure, providing administrators with comprehensive insights into system operations, device actions, and user activities. It connects directly to the Elasticsearch cluster to create interactive dashboards, charts, and monitoring interfaces.

It provides several specialized dashboards tailored to different monitoring needs:

- **System Overview:** Real-time system health metrics including log volume, error rates, and service status across all components
- **Device Monitoring:** PIE device status, connectivity patterns, and performance metrics with geographical distribution if applicable
- **Academic Analytics:** Session participation rates, exam completion statistics, and course-level activity summaries
- **Error Analysis:** Centralized error tracking with severity classification and trend analysis
- **User Activity:** Authentication patterns, session durations, and user engagement metrics

4.2 WEBAPP

In this section, we present a systematic, in-depth account of the technical implementation of the web application. The study commences with an exposition of the system’s overall architecture and directory structure. Subsequently, a detailed analysis is conducted of each functional layer, including internationalization, data access, authentication, user-interface construction, and development tooling. The text places particular emphasis on the rationale behind key design decisions, the trade-offs considered, and the patterns employed to ensure extensibility, maintainability, and security.

4.2.1 Architectural Overview

The application’s organization aligns with the Next.js [6] App Router paradigm, a framework that establishes distinct boundaries between routing, layout, and page logic. Core configuration files such as `next.config.mjs`, `jsconfig.json`, and the environment variable manifest are located at the project root, while source code resides under `/src`, divided into subdirectories for API interactions, UI components, page routes, and static assets. This modular structure enables autonomous development and testing of discrete features, thereby promoting consistency across the codebase.

4.2.2 Middleware

The application provides support both English and Portuguese via URL-based locale prefixes, leveraging the `next-intl` library. Translation resources are maintained as JSON payloads, with one payload designated for each supported language. These payloads are injected during runtime through a custom locale-aware layout. Concurrently, a bespoke middleware layer enforces HTTP security policies, including Content Security Policy, X-Frame-Options, and Cross-Site Scripting (XSS) filters, while seamlessly resolving the user’s preferred locale. This dual-responsibility middleware exemplifies Next.js’s capacity for request-level processing without sacrificing performance.

4.2.3 Data-Access layer

It has been demonstrated that all HTTP interactions are funneled through a centralized client. This client standardizes base URL resolution, header composition, payload encoding, and uniform error handling. Endpoint definitions are canonicalized within an `endpoints.js` manifest, and discrete service modules (e.g. `courseService.js`, `userService.js`) encapsulate operations specific to a given resource. This layered approach decouples business logic from transport concerns and simplifies future adaptations.

4.2.4 Authentication and Authorization

User authentication is implemented via the use of JWT, issued by the backend upon successful credential submission through a login interface driven by the React Hook Form library. Tokens are stored as HTTP-only cookies, thereby mitigating client-side script access, and their validity is assessed on each navigation event. The AuthProvider React context disseminates user identity and authorization state throughout the component tree, thereby facilitating declarative route guards that redirect unauthenticated users to the login flow. This pattern is designed to ensure both security and a consistent user experience. Detailed security implementations are discussed in 5.1

4.2.5 UI Composition and Styling

The user interface is constructed using a bespoke component library that incorporates Shadcn/UI primitives and is styled exclusively with Tailwind CSS's utility-first classes. In the context of web development, global design tokens and CSS resets are typically centralized in a single file known as `globals.css`. The responsive layout and component-level styling adhere to a well-defined design system. The seamless integration of form controls with the validation schema of React Hook Form ensures accessibility compliance and visual consistency across interactive elements, including dialogues, tables, and navigation bars.

4.2.6 Development Workflow and Tooling

A suite of npm scripts—namely, `dev`, `build`, `start`, `lint`, and `format` are present to streamline project maintenance by orchestrating local development, production bundling, static analysis, and automated code styling. ESLint and Prettier configurations are instrumental in enforcing coding standards and minimizing stylistic discrepancies. Environment variables play a crucial role in differentiating between development and production endpoints. The deployment process is streamlined, largely due to the inherent capabilities of Next.js, particularly its native build pipeline and integrated support for environment-specific rewrites.

4.3 DEVAPP

This section provides the implementation overview for the DevApp, the flutter application running on the PIE operating system, and its respective architecture, workflow and configurations. This application was built with the aim of providing the user with an intuitive and simple way of solving the exam, with no need to worry about downloading or submitting the exam, keeping up with the remaining exam time or pre-installing software on their computer.

4.3.1 Architectural Overview

The application contains a few main components: The UI (4.3.2) itself, the subscriber (4.3.3), the logger (4.3.4) and the encryption system. These components work together to create a secure, monitored environment for exam-taking.

The workflow within the application follows these key steps:

1. User authentication through university credentials;
2. Session joining via professor-provided code;
3. Secure downloading and decryption of exam materials;
4. Exam taking with automated periodic submission;
5. Real-time monitoring and notification system, along with professor-managed commands;
6. Final submission and session termination.

Styling

The application uses Flutter’s Material Design with custom theming to ensure a consistent visual identity. The primary color scheme reflects university branding with green as the primary color and orange accents.

A centralized ‘ThemeData’ object defines the primary color, accent color, text styles, and other UI properties for components like buttons and text fields.

To reinforce branding, a custom AppBar displays the PIE logo, a title, and quick access buttons for notifications and URLs. This AppBar is reused across all pages, ensuring a unified look and feel with minimal code duplication.

Configuration

The DevApp employs a centralized configuration system that abstracts environment-specific settings from the application code.

The configuration is loaded from a JSON file stored at `/etc/pie-devapp/config.json`, containing the following key parameters:

- Server API endpoint base URL
- Device identification
- File system paths for exam materials and logs
- Path to the url whitelist file

Additionally, the system maintains a dynamic configuration map that stores session-specific data populated during runtime, such as authentication tokens, course identifiers, and user-specific information. These are accessed through static methods, allowing for easy retrieval and modification without hardcoding values throughout the application.

4.3.2 UI: App Pages

The user interface consists of three main pages, guiding the user through the exam process from start to finish.

Home Page. The Home Page handles authentication and initial setup. Students enter their university credentials and a session code provided by the professor. It includes security features to prevent unauthorized access and shows real-time connection status.

Exam Page. The Exam Page is the main interface during the exam. It supports file-based and URL-based exams, displays a timer for remaining time, and allows manual submission. Notifications and permitted web resources are accessible through a controlled interface.

Finished Page. The Finished Page confirms successful submission and serves as the exit point. It stays connected to the server for potential time extensions, supports notifications, and ensures proper resource cleanup for safe system shutdown.

4.3.3 Subscriber

The Subscriber enables real-time communication between the server and DevApp, supporting instant notifications, remote commands, and session sync. This channel is essential for exam integrity and timely updates.

Architecture

Subscriber uses an event-driven model with Server-Sent Events (SSE) for messages from server to client. It maintains two connections:

- **Global Channel:** For messages to all session participants
- **Student Channel:** For messages to a specific student

This setup allows both group and individual communication efficiently.

Connection Establishment

Subscriber connects to the server, subscribing to two endpoints: the general channel for all students and the student-specific channel for targeted messages.

The subscriber received keep-alive messages to maintain the connection and prevent timeouts. If the connection is lost, it automatically attempts to reconnect after a short delay, ensuring continuous operation.

Connection Management

The Subscriber manages connections with automatic recovery:

- **Connection Monitoring:** Regular checks ensure the server is reachable
- **Automatic Reconnection:** The system reconnects if the connection drops
- **Status Feedback:** Visual indicators show connection status to the user
- **Offline Operation:** Exam features work even if the connection is lost temporarily

This design keeps the exam running smoothly, even during network issues, while maintaining monitoring integrity.

Message Processing

The Subscriber processes several message types that enable remote exam control:

1. **Notifications:** Messages sent by the professor to the student(s) during the exam or general app operation notifications
2. **Exam Start:** Signals to begin the exam, triggering timer and unlocking the environment
3. **Time Extension:** Allows the professor to adjust the exam duration dynamically
4. **Exam Termination:** Forces exam completion due to professor decision or academic integrity concerns
5. **Submission Request:** Triggers immediate submission of current work (without the student knowledge)

4.3.4 Logging

The Logging component is responsible for comprehensive monitoring of client-side activities and system events. It implements both local file-based logging and server-side reporting to ensure complete visibility of all relevant actions during exam sessions.

Event Capture Architecture

The Logger is designed to capture multiple types of system events:

- **Application Events:** Internal state changes, errors, and lifecycle events
- **System Commands:** Shell commands executed during the exam session (bash only, via rsyslog)
- **USB Events:** Insertion and removal of USB devices
- **Network Events:** Changes in connectivity and API interactions
- **File Operations:** Access and modification of exam-related files

Events are captured through a combination of native system monitors, event listeners, and application hooks.

Log Categorization

The logging system implements a structured approach to event categorization based on:

- **Severity Level:** Debug, Information, Warning, Error
- **Event Category:** Network, System, USB, Bluetooth, Command, File
- **Context:** Course, Exam, Session identifiers
- **Timestamp:** Precise event timing

This multi-dimensional categorization enables efficient filtering and analysis of events:

Listing 4.1: Log Structure

```
class Log {
    final DateTime timestamp = DateTime.now();
    final int deviceId = Config.deviceId;
    final String courseId = Config.getFromConfigMap('courseID') ?? Null;
    final int examId = Config.getFromConfigMap('examID') ?? Null;
    final LogLevel level;
    final LogCategory category;
    final String message;
    final Map<String, dynamic> details;

    Map<String, dynamic> toJson() { ... }
}
```

System Command Monitoring

The Logger implements continuous monitoring of system commands through integration with rsyslog:

1. **Command Capture:** rsyslog service is running and capturing into a local log file
2. **File Streaming:** The Logger opens a file stream to read new entries
3. **Event Generation:** Each command is parsed and converted into a Log object

Hardware Event Detection

The Logger implements proactive monitoring of hardware changes, particularly focusing on USB devices:

1. **Initial Inventory:** At startup, all connected USB devices are cataloged
2. **Real-time Monitoring:** The system subscribes to udev events for immediate detection
3. **Device Identification:** Connected devices are identified by vendor, product ID, and characteristics
4. **Change Reporting:** All device-related events are immediately transmitted to the server

Server Synchronization

All captured logs are synchronized with the central server through a dedicated API endpoint, to which the Logger publishes logs in real-time.

Listing 4.2: Log Synchronization

```
static Future<http.Response> post(Log log) async {
  var sessionId = Config.getFromConfigMap('sessionId');
  var nmec = Config.getFromConfigMap('nmec');
  var response = await http.post(
    Uri.parse("${Config.apiUrl}/logs/session/$sessionId/user/$nmec"),
    headers: {
      "Content-Type": "application/json",
      "Authorization": "Bearer ${Config.getFromConfigMap('token')}",
    },
    body: jsonEncode(log.toJson()),
  );

  return response;
}
```

For security purposes, the Logger also locally stores logs in a file at `/var/log/pie-devapp.log` for redundancy and offline access.

4.3.5 Modus Operandi

The DevApp's operational flow is designed to enforce integrity and security throughout the exam process while providing a seamless experience for students. The application follows a structured workflow that ensures proper authentication, exam delivery, monitoring, and submission.

Authentication Flow

The authentication process implements a robust two-stage verification workflow to ensure that only authorized students access the correct exam materials:

1. **Credential Authentication:** Students provide their university credentials (email and password) which are validated against the university's authentication system. These credentials are transmitted securely using HTTPS and processed server-side.
2. **Session Joining:** After successful authentication, students must enter a session code provided by the professor. This code is specific to each exam session and links the student to the correct exam environment.
3. **Environment Configuration:** Upon successful session joining, the application receives and configures exam-specific parameters:
 - Course and exam metadata
 - Exam duration configuration

- Content delivery mechanism (file-based or URL-based)
- Permitted external resources (whitelisted URLs)

The list of permitted URLs (including the exam URL for URL-based exams) is stored in a local file at `/etc/pie-devapp/whitelist.json` to be used by the MITMProxy for filtering network requests during the exam, better described in 5.4.1

Exam Delivery Mechanism

The system supports two distinct delivery mechanisms based on exam requirements:

- **File-based Delivery:** Used for exams requiring local resources such as programming projects, design tasks, or document editing. These exams are downloaded as encrypted files and processed locally.
- **URL-based Delivery:** Used for browser-based assessments like forms, online quizzes, or interactive exercises. These exams launch in a controlled browser environment.

File-based Exams. These exams follow a secure download-decrypt-extract pipeline:

1. The DevApp first obtains the encrypted exam bundle via a secure API endpoint.
2. The downloaded file remains encrypted until exam start is signaled by the professor. Upon receiving the start signal, the system decrypts the content using a session-specific key:

Listing 4.3: Decryption Process

```
final keyBytes = Uint8List.fromList(base64Url.decode(key));
final fileData = await File(encryptedFile).readAsBytes();
final decryptedData = Encryption.decryptFile(fileData, keyBytes);
await File(outputFile).writeAsBytes(decryptedData);
await File(encryptedFile).delete(); // Remove encrypted version
```

3. The decryption utilizes Galois/Counter Mode (GCM) in mode Advanced Encryption Standard (AES), with the key provided by the server at exam start:

Listing 4.4: AES-GCM Decryption

```
static Uint8List decryptFile(Uint8List fileData, Uint8List key, {Uint8List?
  ↪ iv}) {
  iv ??= fileData.sublist(0, 12); // Extract IV from file header
  fileData = fileData.sublist(12); // Get encrypted content

  final gcm = GCMBlockCipher(AESEngine())
    ..init(false, ParametersWithIV(KeyParameter(key), iv));

  return gcm.process(fileData);
}
```

4. For archives (.zip files), the system automatically extracts contents to the exam directory, after decrypting the archive.

URL-based Exams. In the other hand, for URL-based exams, upon receiving the start signal, the application launches the specified URL in an external browser using the `url_launcher` package.

If the exam doesn't automatically open in the browser, the application provides a button to launch it manually.

Exam Solving State

During the exam-solving phase, the DevApp transitions to a background state:

1. **Window Management:** The application transitions from the full-screen lockdown mode to a windowed interface, allowing interaction with required development tools.
2. **Timer Activation:** A countdown timer is initiated, displaying the remaining exam time with decreasing granularity of notifications as time progresses.
3. **Controlled Resource Access:** The system enforces restrictions on permitted resources according to exam configuration:
 - Local file access is limited to the user's home directory (with no root access)
 - Network access is restricted to whitelisted URLs
 - System commands are monitored and logged
 - Device usb connections are monitored
4. **Background Operations:** The application continues to receive notifications, handle commands and automatically save progress.

The exam state concludes via one of three termination paths: student-initiated completion, timer expiration or professor-directed termination, with appropriate safeguards to prevent data loss in each scenario.

Submission Process

However the exam is submitted, whether manually or automatically, the DevApp ensures that all work is securely packaged and transmitted to the server.

The exam's directory's contents are compressed into a zip file, which is then encrypted using the same AES-GCM method as the initial download, ensuring that only the professor can access the submitted work:

Listing 4.5: Submission Process

```
final examDir = Directory(Config.examDirectory);
final examFiles = examDir.listSync(recursive: true);
final zipFile = File('${examDir.path}/submission.zip');
// creation of the zip file ...
final encryptedZipData = encryptFile(zipFile.readAsBytesSync(), keyBytes);
http.MultipartRequest('POST', Uri.parse(submissionUrl))
  ..files.add(await http.MultipartFile.fromPath('submission_file', zipFile.path,
    contentType: MediaType('application', 'zip'),
  ));
```

The encryption function used for submission is similar to the decryption function, but in reverse:

Listing 4.6: AES-GCM Encryption

```
static Uint8List encryptFile(Uint8List fileData, Uint8List key) {
  try {
    final Uint8List iv = Uint8List.fromList(
      List<int>.generate(12, (i) => Random.secure().nextInt(256))
    );
    final gcm = GCMBlockCipher(AESEngine())
      ..init(true, ParametersWithIV(KeyParameter(key), iv));
    final encryptedData = gcm.process(fileData);
    return Uint8List.fromList(iv + encryptedData);
  } catch (e) {
    print('Erro ao encriptar ficheiro: $e');
    return Uint8List(0);
  }
}
```

Exam Completion

Finally, upon successful submission, the DevApp navigates to the Finished Page, which confirms the submission and re-locks the environment. From this point, the application remains connected to the server for potential time extensions or notifications, but no further exam actions are allowed.

4.3.6 Build and Deployment

The DevApp is implemented using Flutter framework, chosen for its cross-platform capabilities, making it easier to create a Linux version for it, and rich set of UI components.

The build process involves compiling the Dart code into native binaries for Linux, ensuring compatibility with the PIE operating system.

In a Debian-based system, the DevApp is compiled with the following commands:

Listing 4.7: Build Commands

```
$ flutter pub get # Fetch dependencies
$ flutter build linux
```

In the directory `build/linux/x64/release/bundle`, the compiled application is located, ready for deployment, just packaging the bundle together.

4.4 OPERATING SYSTEM

In this section, we delve into the process of creating a customized Linux image, providing an overview of the implementation. This includes the selection of the base system, the tools and methodologies used to create the image and the logic behind the chosen approach.

We begin by introducing what the operating system will be used for. After this, it will then be stated which custom image building tools have been found, with a particular focus on the tool known as Debian Live-Build.

This is followed by a discussion of how our image is built. Throughout this section, each component involved in creating the image will be analyzed individually - from the configuration files to the build steps - with technical explanations and implementation decisions clearly presented.

The aim is to provide a clear understanding of how the final image was built, why certain tools or configurations were used and how these choices align with the initial design goals of the system.

Finally, we conclude by addressing aspects of hardware compatibility, ensuring that the resulting image is capable of running on the targeted platforms.

4.4.1 Why we use an operating system

The use of a customized Linux image is justified by the need to create a controlled, customizable and reproducible environment to run on a Raspberry Pi device, which will be used to simulate an isolated system. In addition, the use of an image makes it easier to replicate the environment on multiple Raspberry devices or PEN's, which is essential when testing, developing or demonstrating solutions.

4.4.2 Live-Build

What is Live-Build

Debian Live-Build is an official Debian project tool [7] that allows you to create customized Debian images, known as “Live Systems”. These images can be run directly from devices such as USB sticks. They also offer the option of installing the system on the computer from the live environment.

Live-Build is a set of scripts and tools that automate the process of creating customized live systems. It uses a configuration directory structure to define all aspects of the image to be generated, including:

- System architecture (e.g. amd64, i386);
- Desktop environment (such as GNOME, KDE, XFCE, LXDE);
- Package list management;
- Specific system settings;
- Inclusion of installers (live or full mode).

Why choose Live-Build

Choosing the best tool for creating customized live images basically depends on what you want to do on a technical and control level. So let's make a small comparison between Debian Live-Build and the Linux Live Kit Project, two different approaches to creating live images. The aim is to highlight the differences in approach, customization and the results obtained, helping you to make a decision about which solution to adopt.

Table 4.1 shows the fundamental difference between the approaches of the two tools. As we can see, it makes Live-Build more restricted in terms of compatibility with other distros, but also more controlled, secure and faithful to the Debian ecosystem, which is ultimately what the project requires.

Table 4.1: Approach - Isolated System Image

Features	Live Kit project	Live-Build
Base	Can be used on various distros	Specific to Debian and derivatives (Ubuntu, etc)
Method	Package an existing Linux installation	Build a system from scratch with Debian packages
objective	Transforming an installed system into a live one	Building a live image with total control

Table 4.2 shows that the Live Kit offers a lower learning curve, making it ideal for users who want to package an already functional system. On the other hand, Live-Build requires prior knowledge of Debian packages and structure, but allows for more precise customization through directories, hooks and configuration files. Given that the goal is to build a customized environment in a way that makes it limited, Debian Live-Build becomes the best option

Table 4.2: Customization - Isolated System Image

Features	Live Kit project	Live-Build
Starting Point	A real, working system already installed	Configuration files and packages defined before the build
Easy customization	Very easy if everything is already set up in the system	More complex, requires knowledge of Debian packages and structure
Modularity	Uses optional .sb modules	Use directories and hooks to customize the build

Ultimately both tools generate bootable ISO's, as we can see in table 4.3, but with some significant differences: Live-Build generates a standard Debian image and can include an integrated installer, which facilitates subsequent installations. Live Kit, on the other hand, lacks this functionality by default and requires additional configuration to offer persistence or installation.

Table 4.3: Result - Isolated System Image

Features	Live Kit project	Live-Build
Image type	Bootable ISO, with squashfs + modules	Standard Debian ISO
Persistence	Supported via scripts and boot options	Supported but requires additional configuration
Live + Installer	No installer included by default	You can generate an image with the integrated Debian installer

4.4.3 How it works - Live-Build

Live-Build works on a sequence of commands and a modular directory structure that defines all the stages of the image-building process. The flow usually starts with the **lb config** command, which automatically creates a configuration tree containing all the essential parameters, such as architecture (amd64, i386), file system, package manager, graphical environment and other settings.

After configuration, the **lb build** command starts the actual build process. This command creates a Debian base system within a chroot environment, where packages are downloaded and installed with apt, in isolation from the host system. Afterwards, additional settings are applied, user options are integrated, and finally the system is compressed into a bootable ISO image.

One of the main reasons for using Live-Build is the support for **hooks and includes**, which are basically scripts and files that can be executed at specific stages of the process (for example: before installing packages, after configuring the system and before generating the ISO). These **hooks** allow you to automate customized actions.

In addition, it is possible to manage package lists via **package-lists** files, define system configurations with **config/includes.chroot/**.

4.4.4 Internal structure of our image

Live-Build organizes the configuration of the custom image in two main folders: **auto/** and **config/**. Each has a specific role in the building process, from defining basic parameters to including files and scripts in the final system.

```
.                                #main live-build configuration
|--- auto
|---- config
|   |-- archives                 #package mirrors/repositories
|   |-- hooks                   #extra scripts to run during build stages
|   |-- includes.binary         #files to include on the ISO filesystem
|   |-- includes.chroot        #files to include in the live filesystem
|   |-- includes.installer     #files to include in the installer filesystem
|   |
|   |---- package-lists
|   |   |-- *.list.chroot      #packages to install on the live system
|   |   |-- *.list.binary     #packages to place in the APT repository
|   |
|   |-- packages.chroot        #.deb packages to install on the live system
|   |-- task-lists            #tasksel tasks to install on the live system
|
|-- doc                        #user documentation
|-- Makefile                  #main automation, dependencies management, ...
|-- scripts                   #extra automation scripts
```

The complete configuration of the Live-Build image is attached at the end of the document, in Appendix A.1.

4.4.5 Hardware compatibility

In terms of hardware compatibility, the image created with the Debian live-build depends directly on the target architecture. By default, the live-build generates images for the **amd64** architecture, which covers the vast majority of laptops.

However, when dealing with Apple machines running macOS, the situation presents additional restrictions, mainly due to the closed and proprietary nature of some of Apple’s hardware. On Macs with Intel processors, compatibility with Debian Live images generated via live-build is not usually a problem.

On the other hand, the reality is quite different on Macs with Apple chips (since the M1). These models are based on an **ARM64** architecture, making standard Debian Live images incompatible with this hardware. Although it is technically possible to generate an **ARM64** image using the “--architectures arm64” parameter in lb config, support for Apple hardware in Debian is extremely limited. The image won’t be able to boot correctly, or even if it does, it won’t have functional support for essential components such as the GPU, Wi-Fi, Bluetooth or audio, due to the absence of compatible drivers and firmware in the standard Debian kernel.

Therefore, in the current context, we only generate Debian Live images exclusively for **amd64**, since this architecture is still predominant and has much more robust and consolidated support within the Debian ecosystem. Trying to use Debian Live on **ARM64** Macs requires an adapted kernel, specific bootloaders and, in practice, falls outside the traditional scope of the live-build.

4.5 RASPBERRY PI'S

As an alternative to using PENs USB, and taking into account the scope of the project, the Raspberry Pi Zero W was used as the core of a portable and secure system to support the image previously created for exams. This model was chosen because of its portability, low cost and compatibility when operating in "USB Gadget" mode. In an extended way, this device has been configured to emulate a mass storage device via USB, becoming invisible to the user while providing a customized Linux image to the host computer.

4.5.1 Raspberrys Setup

This subsection describes the steps taken to prepare the Raspberry Pi Zero W as a USB device with gadget mode, capable of presenting the host computer with the image previously customized for exams.

1. The first step was to load the Raspberry Pi OS (Raspbian) on the microSD card intended for the Raspberry Pi Zero W. The official image was recorded using the Raspberry Pi Imager, ensuring that the base system was ready to be modified;
2. After the initial installation, the MicroSD card had two main partitions, and it was necessary to increase the size of the partitions to accommodate the customized .iso image.:
 - A boot partition (/boot);
 - A root partition (/).
3. Next, the Raspberry Pi was configured to operate in USB gadget mode. This involved changes to the boot system and kernel:

```
dtoverlay=dwc2
```

Now, in the cmdline.txt file, the loading of the dwc2 and g_mass_storage modules has been included with the appropriate parameters:

```
modules-load=dwc2, g_mass_storage g_mass_storage.file=/boot/firmware/live-  
→ image-amd64.hybrid.iso g_mass_storage.ro=1 g_mass_storage.removable  
→ =1 g_mass_storage.cdrom=0
```

These changes can be interpreted as kernel-level modifications, since they involve activating modules directly in the boot process and manipulating the system's USB drivers. These steps will be detailed in subsection 4.5.2.

4. Finally, the ISO image was added. This was loaded in the path:

```
/boot/firmware/live-image-amd64.hybrid.iso
```


4.5.2 Gadget Mode

The Raspberry Pi’s “gadget mode” allows the Raspberry Pi Zero W, in this case, to connect to a computer via a USB port, acting as a USB device instead of just as a host. This means that the Raspberry can emulate a mass storage device (USB drivers). This, as a project goal, is particularly useful because it allows you to create another layer of security, such as the development system for running controlled tests.

How it works

This mode is possible thanks to the DesignWare Controller 2 (DWC2) USB driver, present in only a few Raspberry Pi models shown in the table 4.4. By loading the appropriate driver (DWC2) and the gadget modules (`g_mass_storage`, `g_ether`, ...), the Raspberry starts to operate as a mass storage, being recognized by the host computer as if it were ordinary external hardware.

The USB gadget acts at a low level (device layer). In the case of this project, the Pi simulates a USB storage device in read-only mode, presenting a mounted .iso image directly to the host.

Table 4.4: Gadget Mode

Raspberry Pi Models	Compatible with USB Gadget?
Pi Zero	yes
Pi Zero W / WH	yes
Pi Zero 2 W	yes
Pi A / A+	yes
Pi 3A+	yes
Pi 4B	Possible
Pi 400	No
Pi B / B+ / 2B / 3B / 3B+	No

Advantages of Gadget Mode

- **Universal Plug & Play:** the Raspberry can be connected directly to a PC and act as an external device;
- **High customization:** it is possible to control exactly what is presented to the host system;
- **Security:** since the content can be read-only, the system is more resistant to unwanted changes;

Final considerations

The use of gadget mode was a key element in this project, allowing the Raspberry Pi Zero W to be transformed into a direct and secure boot solution. Its flexibility and compatibility with ISO images was essential to ensure that the test environment loaded predictably, without user interaction and without risk of altering the content.

4.5.3 Gadget Mode configuration

The Raspberry Pi Zero W was configured with Gadget Mode by activating the `g_mass_storage` module, specified directly in the kernel command line (`cmdline.txt`) and with support enabled in the `config.txt` file. We chose to use this module because it is simple to apply and takes care of most of the installation and configuration itself. Despite being a simple and effective solution, it does have some limitations, such as not being able to use other gadgets at the same time.

Configuration Files

- `config.txt` - This setting activates the `dwc2` overlay, which is required to enable USB gadget functionality:

```
[all]
dtoverlay=dwc2
disable_splash=1
boot_delay=0
enable_uart=1
```

- `cmdline.txt`:

```
console=serial0,115200 console=tty1 loglevel=5 quiet logo.nologo root=
↪ PARTUUID=04ca6286-02 rootfstype=ext4 fsck.repair=yes rootwait
↪ modules-load=dwc2,g_mass_storage g_mass_storage.file=/boot/firmware
↪ /live-image-amd64.hybrid.iso g_mass_storage.ro=1 g_mass_storage.
↪ removable=1 g_mass_storage.cdrom=0 quiet splash plymouth.ignore-
↪ serial-consoles
```

Here the `g_mass_storage` module is loaded, with the following parameters:

- **file**: path to the ISO image to be displayed as the device;
- **ro=1**: the image is displayed as read-only, preventing any modification by the user or the host system;
- **removable=1**: the device is displayed as "removable", which allows the host system to correctly interpret the drive;
- **cdrom=0**: disables the CD-ROM simulation, in this case it is a regular storage drive.

The technical reference used was the complete Mass Storage Gadget configuration guide [8] and some online forums [9], which provides detailed instructions on the limitations, best practices and modes of operation of the USB gadget, reinforcing the importance of not allowing simultaneous writing to the emulated device.

Security

5.1 AUTHENTICATION AND AUTHORIZATION

The system implements a multi-layered authentication and authorization architecture designed to ensure secure access control across all components, from the backend to the web application and also the Devapp. This section details the security mechanisms employed to protect user accounts, validate identities, and enforce role-based access controls.

The authentication system is built around JWT as the primary authentication mechanism, providing stateless authentication that scales horizontally across the distributed system architecture. The system supports multiple authentication flows to accommodate different client types while maintaining consistent security standards.

5.1.1 JWT Implementation

The JWT implementation is built using the python-jose library with specific configuration choices optimized for security and performance in the academic examination context.

Algorithm and Key Management

The system utilizes the HS256 (Hash-based Message Authentication Code (HMAC) with SHA-256) algorithm for token signing, chosen for several key reasons:

- **Symmetric Key Simplicity:** HS256 uses a single secret key for both signing and verification, simplifying key management in the distributed system architecture;
- **Performance Efficiency:** HMAC-based signing provides faster token generation and validation compared to asymmetric algorithms, crucial for high-frequency API requests during exam sessions;
- **Sufficient Security:** For the system's threat model, HS256 provides adequate cryptographic strength while maintaining operational simplicity;
- **Library Compatibility:** Excellent support across all client implementations.

The secret key is generated using cryptographically secure random generation and stored as an environment variable, ensuring it remains separate from the codebase and can be rotated independently across different deployment environments.

The JWT payload is designed to minimize database queries while providing essential authentication information:

- **Subject (sub):** User's email address serving as the unique identifier across all user roles;
- **Expiration (exp):** Token expiration timestamp for automatic invalidation.

This minimal structure ensures fast token validation while maintaining the necessary information for user identification. Role-based authorization is handled through database queries using the email identifier, providing flexibility for dynamic role changes without requiring token regeneration.

The stateless nature of the JWT implementation supports horizontal scaling of the API servers, essential for handling peak loads during university-wide examination periods while maintaining consistent authentication across all service instances.

5.1.2 Web Application Authentication

The Next.js web application implements a custom authentication solution built around JWT tokens and secure cookie management. When users authenticate through the login form, credentials are validated against the FastAPI backend, which returns a JWT token upon successful authentication. This token is then stored in a secure HTTP-only cookie with comprehensive security attributes.

Cookie security is implemented through multiple layers of protection. The authentication cookies are configured as HTTP-only to prevent client-side JavaScript access, significantly reducing the risk of XSS-based token theft, cookies are marked as Secure to ensure transmission only over HTTPS connections. The *SameSite=Strict* attribute provides CSRF protection by preventing cross-site request inclusion of authentication cookies.

Session management includes automatic token expiration monitoring through client-side JWT payload inspection. The system calculates token expiration times and implements automatic refresh mechanisms approximately five minutes before token expiry. This approach maintains user sessions without requiring frequent re-authentication while ensuring expired tokens are promptly invalidated.

Role-based access control restricts administrative interface access to users without admin roles.

5.1.3 Device Authentication

PIE devices implement a specialized authentication flow that combines user-based JWT authentication with device identification for secure exam participation and monitoring.

Device Registration

Each PIE device is configured with a unique device identifier stored in a configuration file. The registration process includes:

- **Static Device ID:** Each device maintains a unique integer identifier configured during deployment;
- **API Endpoint Configuration:** Devices are configured with the specific API base URL for their deployment environment;
- **Administrative Registration:** Device IDs must be registered in the system database before they can participate in exam sessions.

User Authentication Flow

The device authentication process follows a two-stage approach combining user credentials with device identification:

Stage 1: User Authentication. Students authenticate using their university credentials through the 4.3 login interface. The authentication request includes:

- **Email and Password:** Student Credentials validated against the central authentication system;
- **Device Context:** The device ID is included in authentication requests to establish device-user association;
- **JWT Token Generation:** Successful authentication returns a JWT token containing user identity and permissions.

Stage 2: Session Registration. Following successful user authentication, devices must register for specific exam sessions:

- **Join Code Validation:** Students enter a session-specific join code provided by the professor;
- **Session-User Association:** The system creates a session-user relationship linking the authenticated user, device, and exam session;
- **Session Context Storage:** Device stores session metadata including session ID, exam ID, course ID, and duration locally.

Token Management and Storage

Device authentication tokens are managed through a secure local storage mechanism:

- **Local Configuration Storage:** Authentication tokens and session data are stored in the device's configuration map during runtime;
- **Session Persistence:** Authentication state persists throughout the exam session duration;
- **Automatic Cleanup:** All data is cleared when devices are reset.

5.1.4 Authorization Framework

The system implements Role-Based Access Control (RBAC) with three distinct privilege levels, each designed to provide appropriate access while maintaining the principle of least privilege.

The system implements a three-tier role hierarchy designed to provide appropriate access levels while maintaining security boundaries between different user types.

Student Role

Students have controlled access focused on examination participation and submission activities. They can access courses in which they are enrolled and participate in active exam sessions within those courses. Their primary capabilities include submitting files during assigned sessions, accessing course materials and exam instructions in read-only mode, and participating in screen sharing when requested by instructors. Students cannot access administrative functions or other students' data.

Professor Role

Professors have comprehensive management capabilities within their assigned academic domains. They possess full course management rights, including the ability to create, modify, and control exam sessions. During active sessions, professors can monitor student activities in real-time, control screen sharing sessions, and review submitted work. They have access to course-specific analytics including student participation data and performance metrics. Additionally, professors can manage PIE devices assigned to their classrooms and monitor device status during examinations.

Administrator Role

Administrators maintain system-wide privileges across all platform components. They can manage user accounts, course assignments, and global system configuration. Their responsibilities include device registration and monitoring across the entire institution, comprehensive access to system logs and analytics through the Kibana dashboard, and security monitoring with incident response capabilities. Administrators can configure system components and maintain overall platform security and performance.

5.1.5 Password Security

The system implements industry-standard password security practices centered around bcrypt hashing with comprehensive validation mechanisms to protect user credentials against common attack vectors.

Password Hashing Implementation

Password security is built around the bcrypt algorithm with default configuration optimized for the academic examination context. The implementation utilizes the standard bcrypt library to generate cryptographically secure password hashes with automatic salt generation for each password.

The system uses bcrypt's default work factor of 12 rounds, providing computational resistance against brute force attacks while maintaining reasonable authentication response times during peak examination periods. Each password undergoes individual salt generation using bcrypt's built-in 'gensalt()' mechanism, ensuring that identical passwords produce different hash values across different users.

Password verification follows secure comparison practices using bcrypt's constant-time 'checkpw()' function, ensuring that verification timing remains consistent regardless of password correctness to prevent timing attacks.

Hash Storage and Management

The password storage mechanism implements several security measures to protect stored credentials. Hash values are stored in the MySQL database with appropriate field constraints to prevent truncation or corruption. The system includes validation logic to verify hash format integrity before attempting verification operations, preventing potential security vulnerabilities from malformed hash data.

Database-level security measures protect stored password hashes through appropriate access controls and encryption at rest. The system ensures that password hashes are never exposed through API responses or logging mechanisms, maintaining strict separation between authentication credentials and application data flows.

Authentication Flow Security

The authentication process incorporates multiple security layers to prevent credential compromise. User login attempts undergo rate limiting to prevent brute force attacks, though the specific implementation details vary by client type. The system maintains audit logs of authentication attempts, recording both successful and failed login events for security monitoring purposes.

Device Security Features

The PIE devices implement several security measures to maintain exam integrity:

- **Keyboard Interception:** Comprehensive keyboard event handling prevents unauthorized key combinations and system access through common keybinds;
- **Fullscreen Lock:** Application maintains fullscreen mode with window manager controls to prevent system access before session start;
- **System Command Monitoring:** Real-time monitoring and logging of system commands executed during exam sessions;
- **USB Device Detection:** Automatic detection and logging of USB device connections for security monitoring.

This device authentication framework ensures that only authorized users on registered devices can participate in exam sessions, while maintaining comprehensive logging and monitoring for security and academic integrity purposes.

5.2 COMMUNICATIONS SECURITY

The communications between all the components of the system contain security features, ensuring data integrity and confidentiality. Security measures are customised according to the specific characteristics of each communication type and its respective requirements.

5.2.1 Transport Layer Security

All network communications utilize Transport Layer Security with HTTPS enforcement across all endpoints. The API deployment implements comprehensive TLS encryption for communications, while the web application ensures secure client-server interaction.

Certificate management follows industry best practices with automated certificate provisioning and renewal through the deployment service.

5.2.2 API Communication Security

Request validation utilizes Pydantic models to ensure data integrity and prevent injection attacks through malformed payloads. The system implements structured error handling that provides meaningful feedback to legitimate clients while avoiding information disclosure that could assist potential attackers. Response headers include security directives such as Content-Type validation and Cross-Origin Resource Sharing (CORS) policies configured to restrict cross-origin access to authorized domains only.

Rate limiting and request throttling mechanisms protect against denial-of-service attacks and brute force authentication attempts. The implementation limits the number of requests made in a specific time frame using the `slowapi` [10] library.

5.2.3 Real-time Communication Security

WebSocket connections for screen sharing functionality implement JWT-based authentication before connection establishment. The system validates tokens passed as query parameters during WebSocket upgrade, ensuring only authorized users can participate in real-time communications.

The WebRTC signaling process utilizes Redis Pub/Sub channels for message routing between participants. Each signaling session uses session-specific and user-specific channel identifiers to maintain communication boundaries. The WebRTC implementation utilizes built-in Datagram Transport Layer Security (DTLS) encryption for all peer-to-peer media streams, providing end-to-end security for examination content without additional configuration.

Session isolation is maintained through session ID validation in WebSocket messages, ensuring that screen sharing requests and responses are properly scoped to their designated exam contexts. The hybrid architecture enables direct peer-to-peer connections after initial signaling through the server, reducing server load while maintaining security through WebRTC's inherent encryption.

5.2.4 Database Communication Security

SQL injection prevention is achieved through SQLAlchemy's parameterized query system, ensuring that user input never directly interpolates into SQL statements. The ORM layer provides additional validation and sanitization of data before database operations, maintaining data integrity at multiple levels.

5.2.5 Redis Communication Security

Redis Pub/Sub and Streams communications implement authentication and channel isolation to prevent unauthorized access to real-time messaging infrastructure. Each Redis connection requires authentication credentials configured through environment variables, with different access levels for different service components.

Channel naming conventions include session-specific identifiers and expiration policies to prevent data persistence beyond intended session lifetimes. Message validation ensures that only properly formatted communications are processed, with malformed messages logged and discarded to prevent system disruption.

The Redis infrastructure implements memory usage limits and connection throttling to prevent resource exhaustion attacks.

5.3 STORAGE

The storage security architecture encompasses comprehensive protection mechanisms for both file-based content and database information, ensuring data integrity, confidentiality, and availability throughout the evaluation lifecycle. The system implements multiple layers of security controls across file storage, database operations, and configuration management to protect sensitive academic content and student information against various security threats.

5.3.1 File Storage Security

The file storage system operates through a centralized File Manager component that enforces consistent security policies across all file operations. This centralized approach ensures that all uploaded content, including examination materials and student submissions, undergoes rigorous security validation before permanent storage.

File Validation and Content Security

The system implements comprehensive file validation mechanisms that operate at multiple levels to prevent malicious content from entering the storage infrastructure. File uploads undergo extensive validation including Multipurpose Internet Mail Extension (MIME) type verification against file extensions, file size constraints to prevent resource exhaustion, and filename sanitization to eliminate path traversal vulnerabilities.

The validation process examines file headers to verify authentic file types, preventing disguised executable files from being stored as academic content. The system supports specific file types relevant to academic contexts:

- **Document formats:** PDF documents and Microsoft Word documents (DOC/DOCX)
- **Image formats:** PNG and JPEG images
- **Archive formats:** ZIP compressed archives
- **Programming source files:** Java, Python, C, C++, and header files

Each file type undergoes specialized validation tailored to its expected content structure and security characteristics.

Filename sanitization removes dangerous characters and path components that could enable directory traversal attacks. The sanitization process converts non-ASCII characters using transliteration, replaces special characters with underscores, and ensures that filenames cannot reference parent directories or system paths. Additionally, the system enforces maximum file size limits of 10MB for individual files and 50MB for extracted ZIP contents to prevent resource exhaustion attacks.

Malware Detection and Prevention

The storage security framework integrates ClamAV [11] antivirus scanning to detect known malware signatures in uploaded files. The malware detection system operates through network connections to a dedicated ClamAV daemon container, ensuring that virus definitions remain current and scanning performance does not impact API response times.

Beyond signature-based detection, the system implements pattern-based security scanning that identifies potentially malicious content through header analysis. The scanning process examines file headers for executable patterns that could indicate malicious intent:

- **Windows PE headers:** MZ signature indicating Windows executable files
- **ELF binaries:** Unix/Linux executable file format headers
- **Shell script indicators:** Bash and shell script execution headers
- **Web script patterns:** PHP opening tags and server-side scripting content
- **Java bytecode:** Compiled Java class file signatures
- **Windows batch scripts:** DOS batch file commands and PowerShell indicators

The pattern detection system makes exceptions for legitimate programming source files, allowing educational content in supported programming languages while blocking potentially dangerous executable content.

For ZIP archives, the system implements specialized validation that prevents ZIP bomb attacks through compression ratio analysis, limits the number of files within archives to prevent resource exhaustion, and recursively validates nested archives to detect multi-layered compression attacks. The recursive validation includes depth limits to prevent infinite recursion scenarios while maintaining security effectiveness.

File Encryption and Key Management

Examination materials undergo mandatory encryption before storage using AES-GCM encryption with 256-bit keys. The encryption implementation utilizes the cryptographically secure AES-GCM cipher from the Python cryptography library, ensuring both confidentiality and integrity protection for stored academic content.

Each examination session receives a unique encryption key generated using the `secrets.token_bytes(32)` function, which produces 32 bytes (256 bits) of cryptographically secure random data. The key generation process utilizes the operating system's cryptographically secure random number generator to ensure entropy quality appropriate for cryptographic applications. Encryption keys are encoded using URL-safe base64 encoding for database storage while maintaining separation from encrypted content.

The encryption process generates unique 12-byte nonces for each file using `secrets.token_bytes(12)`, preventing nonce reuse attacks that could compromise encryption security. The nonce is prepended to the encrypted content, creating a self-contained encrypted file that includes all necessary decryption parameters while maintaining security boundaries.

When multiple files are uploaded for a single session, the system creates **zip!** archives before encryption, reducing storage overhead while maintaining individual file integrity. The archival process preserves original filenames and directory structures while ensuring that the final encrypted package can be efficiently distributed to authorized participants.

Access Control and Path Security

File access control operates through multiple authentication and authorization layers that verify user permissions before allowing any file operations. The system implements role-based access control where professors can upload and manage examination materials for their designated courses, while students can only download materials for active sessions in which they are registered participants.

Path security is enforced through secure path construction functions that prevent directory traversal attacks and ensure that all file operations remain within designated storage boundaries. The `secure_join_path` function sanitizes path components and validates that constructed paths remain within the authorized storage hierarchy.

Storage organization follows a hierarchical structure that reflects database relationships while maintaining security boundaries between different academic contexts. Files are organized using structured directory hierarchies based on sanitized course names, examination identifiers, and session information, ensuring consistent organization and simplified access control enforcement.

Directory creation includes appropriate permission settings that restrict access to authorized system users while preventing unauthorized system access. The `create_secure_directory` function sets directory permissions to `0o750`, providing read and execute permissions for the owner and group.

Connection Security and Authentication

Database connections utilize SQLAlchemy's connection pooling mechanism to manage database connectivity efficiently while maintaining connection security. Database credentials are managed through environment variables and secure configuration management, ensuring that sensitive connection information remains separate from application code and version control systems.

The database operates within the containerized environment using Docker networking, which provides network isolation between the database container and external networks while allowing authorized API service connections.

Query Security and Injection Prevention

The system implements comprehensive SQL injection prevention through SQLAlchemy's parameterized query system, ensuring that user input undergoes proper sanitization before database operations. The ORM layer provides additional validation mechanisms that verify data types and constraints before executing database queries, maintaining data integrity at multiple levels.

Database operations undergo privilege validation to ensure that users can only perform operations appropriate to their role level. The authorization framework validates user permissions through JWT token validation before allowing database access, ensuring that all database operations maintain proper security context.

All database queries utilize SQLAlchemy's ORM layer, which automatically parameterizes queries and prevents direct SQL injection through string interpolation. The system does not construct raw SQL queries from user input, maintaining security through the abstraction layer.

Data Integrity and Validation

Database schema design includes comprehensive data validation constraints that prevent invalid data entry and maintain referential integrity across related tables. The schema utilizes foreign key constraints, data type validation, and business logic constraints to ensure that stored information maintains consistency and accuracy.

The ORM layer implements additional validation through Pydantic models that verify data formats and business logic before database operations. This dual-layer validation approach ensures that invalid data cannot enter the database through any application pathway while providing clear error messages for legitimate validation failures.

5.4 OPERATING SYSTEM SECURITY

Operating system security plays a key role in ensuring a reliable, secure and controllable assessment environment.

5.4.1 Internet Access Control

Given that the exams are held on a system to which the student connects their personal computer, it was imperative to configure this environment in such a way as to prevent any kind of unauthorized access to the internet, as well as preventing external communications that could compromise the integrity of the exam. To this end, a proxy system with an access whitelist was implemented using **mitmproxy** [12] as we can see in A.2.

The operating system, based on Debian created with Live-Build, has been modified to ensure that all external connections must pass through a **local proxy** - more specifically, **mitmdump**, a component of mitmproxy - configured to listen on port 8081. The aim is to restrict access only to permitted **URL**s previously defined by the teacher, uploaded during the creation of the exam session.

Starting firefox and mitmproxy

To achieve this control, it was necessary to ensure that the Firefox (.mozilla) and mitmproxy (.mitmproxy) configuration directories were correctly initialized during system startup. Boot hooks were created, using scripts inserted into the live-build configuration, which automate this process:

- **Starting Firefox in headless mode** to force the creation of the .mozilla directory, which is essential for later certificate injection;
- **Running mitmdump temporarily** to generate the Certificate Authority (CA) certificate file (mitmproxy-ca-cert.pem) in the .mitmproxy directory.

Certificate Injection in Firefox

Once the mitmproxy CA certificate has been generated, an additional script has been created in order to import these certificates into all existing Firefox profiles. This process uses the certutil tool and checks the existence of the profiles.ini file and the .default* profiles before proceeding with the import. In this way, it was possible to ensure that HTTPS communications monitored by mitmproxy did not generate security alerts in the browser, thus maintaining a smooth and secure user experience.

Proxy configuration in Firefox

In order for all HTTP and HTTPS traffic to be correctly channeled to mitmdump, the browser was manually configured to use the 127.0.0.1 proxy on port 8081. This configuration was applied via the `proxy.cfg` and `autoconfig.js` files, which were inserted into the default Firefox directories during the creation of the image (live-build), namely in:

```
/usr/lib/firefox-esr/defaults/pref/autoconfig.js
/usr/lib/firefox-esr/proxy.cfg
```

Through the use of **lockPref()** functions, these settings are enforced and cannot be changed by the end user, thus ensuring that traffic cannot bypass the defined proxy.

Dynamic Whitelist Script

The `whitelist.py` file, used as a control script by mitmdump, explicitly defines which domains and URLs are allowed. It includes dynamic verification logic that periodically reads a text file (`/etc/iptables/whitelist.txt`) to update the list of authorized accesses. In addition, it analyzes referers to allow dependent resources to be loaded without the need to list all URLs manually. Any attempt to access URLs not included in the list results in a 403 response - “Access to url blocked by PIE” - effectively blocking access.

This approach ensures that students have access exclusively to the educational resources defined by the teacher (e.g. Moodle resources), while any other websites or services are blocked.

5.4.2 User and Privilege Management

In order to guarantee a secure and controlled environment for carrying out assessments, a restrictive user and permission policy has been defined within the operating system. This measure aims to prevent students from executing commands with administrative privileges, as well as providing a secure maintenance and testing channel through a duly authorized admin user.

User Structure

The operating system contains two main users:

- **student:** Default user used by students when taking exams;
- **admin:** User reserved for authorized maintenance and testing, with administrative permissions.

These users are created automatically during the image build process, via a specific hook included in the chroot phase of the live-build. The respective user creation script can be found at:

```
/pie-system-image/config/hooks/live/0100-add-user.hook.chroot
```

With the content:

```
set -e
useradd -m -s /bin/bash student
echo "student:pie" | chpasswd

useradd -m -G sudo,adm -s /bin/bash admin
echo "admin:admin" | chpasswd
```

The student user is created without any superuser permissions. On the other hand, the admin user is added to the sudo groups, giving them privileged access for system maintenance and administration purposes.

Restrictions for the user “student”

In order to strengthen security, the user student is prevented from executing any commands with sudo. When attempting to do so, the system responds with the standard error message:

```
"student is not in the sudoers file. This incident will be reported."
```

The absence of administrative privileges for the student user prevents any modification to the operating system, installation of software, access to protected files or execution of potentially malicious scripts. This approach ensures that the integrity of the assessment environment is maintained throughout the session.

"Admin" user for Maintenance

The admin user is intended exclusively for testing and maintenance tasks by authorized personnel. Inclusion in the sudo and adm groups gives you full access to the system, so you can perform operations such as:

- System updates;
- Package installation;
- Accessing and reading system logs;
- Changes in the configuration of the network environment, proxy or sensitive files.

This user is not accessible by students during exams, ensuring that only accredited operators have access to administrative functions.

5.5 RASPBERRY VS USB PENDRIVE

One of the greatest innovations of our project is the use of the Raspberry Zero W configured with gadget mode as an isolated and controlled storage device, offering a new layer of security that is significantly superior to any traditional solution based on a simple USB flash drive.

One of the biggest differences between the Raspberry Zero W applied with gadget mode and a traditional USB flash drive is that the former, when applied with gadget mode, behaves like an active element, with internal logic and computational capacity, while the flash drive is a passive device, which only stores data without performing any kind of processing.

5.5.1 Active Element vs Passive Element: Fundamental Differences:

Passive Device - USB Pen Drive

A USB flash drive is a passive device. This means that it has no significant processing capacity or internal logic. Its main function is to store data persistently and respond to basic read and write commands sent by the host computer. Technical characteristics of the passive device:

- **No operating system:** The flash drive does not run any operating system or control software. It is basically a flash drive with a simple USB interface controller;
- **No protection against manipulation or corruption:** The flash drive cannot prevent data from being corrupted or deleted, or malware from modifying the files;
- **Vulnerable to external attacks:** As it has no logic to monitor or protect data, it can be easily attacked, for example by executing malicious code on the host that alters the contents of the flash drive, or by attacks on the firmware;
- **Easy mounting in virtual machines and multiple systems:** It is trivial to mount the flash drive in virtual environments for analysis or unauthorized copying of data.

Active Device - Raspberry Pi in USB Gadget Mode

The Raspberry Pi configured in USB gadget mode is an active device. This means that it is a complete computer capable of running an operating system and customized software, able to manage and strictly control all the operations it performs. Technical characteristics of the active device:

- **Resilience to firmware attacks:** The Raspberry Pi does not use fixed USB firmware in the traditional sense. Instead, the operation of the USB gadget is entirely controlled by the Linux operating system running on the Pi, via kernel modules (such as `g_mass_storage` or `libcomposite`, our case is `g_mass_storage`). To compromise the firmware, an attacker would have to compromise the Linux operating system on the Pi, which is protected by the immutable image and system integrity mechanisms;

- **Complex and dynamic operation:** As an active device, the Pi can adapt the behavior of the USB gadget as needed, featuring multiple USB functions, read-only modes, among others;
- **The operating system image is not directly exposed to the host:** Unlike a USB flash driver, where the USB device directly presents the files and the file system structure to the host computer, on the Raspberry Pi the system image is embedded internally in Raspbian itself. In other words, the host never has direct access to the operating system image or its internal contents, only to the virtual USB device presented by the gadget, controlled by the Pi;
- **No unauthorized access to the image:** As the Raspberry Pi manages the backing store and USB interface internally, the host cannot mount or modify the operating system image directly. This prevents attacks aimed at modifying the system to compromise security, which is common with passive devices;
- **Enhanced protection against manipulation:** The system image is loaded onto an immutable partition (Live-build image), ensuring that the operating environment remains consistent and clean between sessions, even after abrupt shutdowns.

5.5.2 Raspberry vs USB Pendrive

Table 5.1: Raspberry Pi vs USB Pendrive - Security Level

Security Aspect	Raspberry Pi	USB Pendrive
Environment Isolation	Immutable operating system image embedded in the device, isolated environment	No isolation, direct access to the file system
Firmware Attack Resistance	Fixed firmware and system monitoring against tampering	Reprogrammable firmware
Prevention of Unauthorized Mounting	Host has no direct access to the system image, only to the controlled device	Easy mounting and reading on any system or VM

5.5.3 Conclusion

The use of the Raspberry Pi configured in USB gadget mode introduces a fundamentally stronger and more versatile security layer compared to traditional USB flash drives. By operating as an active device with internal logic, a controlled and immutable operating environment, and robust protections against unauthorized access and manipulation, the Raspberry Pi ensures the integrity and confidentiality of sensitive data in scenarios where security is paramount. This extra layer of security significantly reduces risks associated with data corruption, unauthorized access, and firmware attacks—challenges that simple passive USB flash drives cannot adequately address. Thus, leveraging the Raspberry Pi in this manner represents a major advancement in secure digital storage solutions, particularly in critical applications such as academic assessments.

Tests and Results

6.1 TESTS

Due to the short development time available for the project, unfortunately we were unable to perform a set of formal usability tests on the latest version of the project. However, we had a great opportunity to test the work during Students@DETI, an event consisting of a fair showcasing various projects developed by DETI-UA students. Here we had the opportunity to put our work to the test with different colleagues and professors from the university. All the feedback we received from both professors and students was very positive, praising mainly the basic concept of the work and its intuitiveness.

6.2 RESULTS

At the inception of the project, five primary objectives were delineated, which were pursued throughout the course of its development. With the project's conclusion, it is with great satisfaction that we affirm the successful realization of the objectives established at the project's inception.

6.2.1 Secure and Controlled Exam Environment

The major distinguishing feature of the project was the creation of an exam environment that is truly secure for both the professor and the student taking the test on their laptop. This environment is achieved through the combination of factors developed during the project, namely, the combination of effective monitoring by the professor and the creation of a completely isolated environment using the resources provided by Raspberry Pi and the image developed throughout the project, making our environment currently one of the safest for conducting exams.

6.2.2 Real-Time Monitoring and Support

While this objective remains challenging and room for improvement exists, it can be regarded as fulfilled due to the supplementary layers of assistance and environmental monitoring that our work has contributed to the existing solutions. These include real-time broadcast messages from the instructor to the student, enhanced session time management, and the logging system accessible to the instructor.

6.2.3 Usability and Comfort

This is an area in which our work has a significant positive impact. We provide teachers with a familiar and intuitive interface for creating and monitoring tests while their students can take their assessments on their own laptops without the possibility of cheating.

6.2.4 Portability and Affordability

Our project stands out for its portability, whether with the USB flash drive solution or the Raspberry Pi solution, both of them are really easy to use and small to carry around.

We consider our project to be quite affordable, compared to the price and maintenance costs of desktop computers in different university rooms, our solutions are much cheaper and more scalable. If we are going to produce our final product for a larger audience, the idea is to create a "custom-made Raspberry Pi" with only the necessary components, which would decrease the production price significantly.

6.2.5 Simplified Exam Management for Teachers

The entire development of the WebApp focused primarily on the user experience of professors leading us to believe that we have achieved a better solution than what is currently offered by both Moodle and SEB. We have managed to condense everything a professor needs and desires into a single application, from the process of creating an exam to the phase of collecting students' answers.

Future Work

In order to extend and strengthen the capabilities of the proposed remote Exams management platform, several key avenues of further investigation and development are identified:

7.1 SECURE REAL-TIME STUDENT SCREEN-SHARING

The objective is to design and implement a robust, end-to-end encrypted screen-sharing module that enables instructors to monitor student workstations in real time while preserving user privacy and complying with institutional security policies.

7.1.1 Objectives

- **End-to-end confidentiality:** Guarantee that all screen content is encrypted in transit and at rest, preventing eavesdropping or tampering.
- **Privacy-aware sampling:** Allow instructors to view only designated application windows or desktop regions, avoiding exposure of personal data.
- **Low-latency delivery:** Maintain interactive frame rates over constrained networks to support live supervision during assessments.

7.2 FRAUDULENT-ACTIVITY ALERTING FRAMEWORK

To enhance the integrity and reliability of the remote classroom management platform, we propose the design and implementation of an Intelligent Logging and Fraudulent-Activity Alerting Framework. This subsystem will not only capture detailed operational events but will also apply advanced analytic to distinguish legitimate system behaviors from potentially fraudulent or anomalous activities.

7.2.1 Objectives

- **High-Fidelity Event Capture:** Instrument both client and server components to record fine-grained logs of user actions
- **Real-Time Alerting Pipeline:** Construct a event stream feeding into a real-time processing engine, which triggers alerts when suspicious patterns are detected (e.g terminal usage).
- **Teacher Notification Interface:** Design an intuitive dashboard and push-notification mechanism (e.g. in-app toast) that summarizes flagged incidents and provides contextual evidence to facilitate rapid instructor adjudication.

7.3 PLUG-GABLE AUTHENTICATION VIA EXTERNAL IDENTITY PROVIDERS

Refactor both the WebApp front end and the Client service to adopt an identity authentication layer, thereby allowing seamless integration with arbitrary institutional identity providers.

7.3.1 Objectives

- **Identity-agnostic login:** Enable students and staff to authenticate using their institution's existing credentials.
- **Token interoperability:** Issue access tokens compatible with both WebApp and lightweight Client daemons.
- **Revocation and session management:** Support real-time token revocation and single-logout flows.

7.4 BIOS-LEVEL BOOTLOADER INTEGRATION

Investigate methods to automate or bypass manual Basic Input/Output System (BIOS) configuration—potentially through Unified Extensible Firmware Interface (UEFI) scripting or PXE-based boot provisioning—so that student devices can boot directly into the management environment without end-user intervention.

7.4.1 Objectives

- **Zero-touch provisioning:** Eliminate manual BIOS/UEFI configuration by automating boot settings.
- **Cross-platform support:** Accommodate both legacy BIOS and modern UEFI firmware.
- **Secure boot compliance:** Ensure the boot sequence's integrity through cryptographic validation.

7.5 CENTRALIZED, REMOTE DEVICE MANAGEMENT CONSOLE

Develop a scalable, web-based administration portal that offers centralized inventory, configuration, software deployment, and health-monitoring for all enrolled endpoints, leveraging modern micro services and real-time messaging architectures.

7.5.1 Objectives

- **Unified endpoint view:** Provide real-time telemetry on hardware health, OS status, and software inventory for all managed devices.
- **Policy enforcement:** Apply configuration baselines, software updates, and security patches at scale.
- **Interactive control:** Offer remote shell access, file transfer, and screen-view capabilities as needed.

Referências

- [1] M. Hillier, *Transforming exams across Australia : processes and platform for e-exams in high stakes, supervised environments*. Australian Government Department of Education and Training, 2019, p. 44, ISBN: 9781760517564.
- [2] *Seb - safe exam browser*.
- [3] D. Cruz, D. Mortágua, L. Sousa, and T. Oliveira, “Eel-electronic exam lab,” Tech. Rep., 2021.
- [4] *Fastapi*. [Online]. Available: <https://fastapi.tiangolo.com/>.
- [5] *Getting started with web rtc*. [Online]. Available: <https://webrtc.org/getting-started/overview>.
- [6] *Next.js documentation*. [Online]. Available: <https://nextjs.org/docs>.
- [7] Debian Live Project, *Building a custom debian iso image: Using the default configuration*, 2025. [Online]. Available: <https://debian-live-config.readthedocs.io/en/latest/custom.html#build-using-the-default-configuration>.
- [8] Unknown Author, “Usb mass storage gadget: A beginner’s guide,” Tech. Rep., 2021, Licenciado sob Creative Commons CC BY-NC-SA 4.0.
- [9] W. Walter, *How fast can a Raspberry Pi Zero 2w boot?* 2022. [Online]. Available: <https://medium.com/@awarlt12207/how-fast-can-a-raspberry-pi-zero-2w-boot-8614de1cbd64>.
- [10] “Slowapi,” [Online]. Available: <https://slowapi.readthedocs.io/en/latest/>.
- [11] Cisco Talos Intelligence Group, *Clamav: Open-source antivirus engine for detecting malware*, 2025. [Online]. Available: <http://www.clamav.net/>.
- [12] *Introduction*. [Online]. Available: <https://docs.mitmproxy.org/stable/>.

Additional content

A.1 LIVE-BUILD CONFIG

```
pie-system-image/
|--- Makefile                                # Automatiza os comandos de build
|----- config/                             # Diretorio principal
|   |--- binary
|   |--- bootstrap
|   |--- chroot
|   |--- common                             # Configuracoes comuns a varias fases
|   |--- source                             # builds de codigo-fonte
|   |----- hooks/                          # Scripts executados durante a build
|       |----- chroot/                     # Personalizacoes
|           |--- 9997-setup-wifi.sh
|           |--- 9998-setup-mitmproxy-student.sh
|           |--- 9999-setup-firefox-student.sh
|           |
|           |----- live/                    # Hooks para configuracao do ambiente
|               |--- 0100-add-user.hook.chroot
|               |--- 0200-create-desktop-dir.hook.chroot
|               |
|               |----- normal/               # Fase de personalizacao pos-instalacao
|                   |--- 0010-install-vscode.hook.chroot
|                   |--- 8020-remove-dbus-machine-id.hook.chroot
|                   |--- ... (Outros scripts de limpeza e setup)
|                   |--- enable-services.chroot
|           |
|   |----- includes.chroot/                 # Ficheiros inseridos no sistema
|       |----- etc/                         # Configuracoes do sistema
|           |--- rc.local
|           |--- sysctl.conf
|           |--- lightdm/lightdm.conf
|           |--- pie-devapp/config.json
|           |--- rsyslog.d/bash_commands.conf
|           |----- skel/                     # Perfil default de utilizadores
```

```

| | | | |--- .bashrc
| | | | |---- .config/
| | | | |---- autostart/disable-icons.desktop
| | | | |
| | | | |----- systemd/system/      # Servicos de arranque automatico
| | | | |--- devapp.service
| | | | |--- multi-user.target.wants/
| | | | |--- inject-certs.service
| | | | |
| | | | |----- xdg/autostart/        # Autostart via ambiente grafico
| | | | |--- mitmproxy.desktop
| | | | |
| | | | |----- opt/devapp/           # Aplicacao personalizada incluida
| | | | |--- devapp
| | | | |--- flutter_assets/...
| | | | |--- lib/libapp.so
| | | | |
| | | | |----- usr/local/bin/        # Scripts e utilitarios locais
| | | | |--- init-firefox.sh
| | | | |--- whitelist.py
| | | | |--- share/images/desktop-base/ # Fundos e icones personalizados
| | | | |
| | | | |----- package-lists/        # Listas de pacotes a instalar
| | | | |--- 0001-wifi.list.chroot
| | | | |--- 0002-live.list.chroot
| | | | |--- 0003-development.list.chroot
| | | | |--- 0004-pkgs.list.chroot
| | | | |--- 0005-whitelist.list.chroot
| | | | |--- 0006-logs.list.chroot
| | | | |--- live.list.chroot

```

A.2 WHITELIST.PY

```
from mitmproxy import http
from urllib.parse import urlparse, urljoin
import os
import time
import sys

WHITELIST_FILE = os.path.expanduser("/etc/iptables/whitelist.txt")
WHITELIST_DOMAINS = { }
MANUAL_WHITELIST_URLS = { }
LOADED_WHITELIST_URLS = set(MANUAL_WHITELIST_URLS)
allowed_referers = set()

LAST_CHECK_TIME = 0
CHECK_INTERVAL = 30

def is_whitelisted_domain(url):
    parsed_url = urlparse(url)
    return parsed_url.hostname in WHITELIST_DOMAINS

def reload_whitelist():
    global LAST_CHECK_TIME, LOADED_WHITELIST_URLS

    current_time = time.time()
    if current_time - LAST_CHECK_TIME < CHECK_INTERVAL:
        return
    LAST_CHECK_TIME = current_time
    try:
        updated_whitelist = set(MANUAL_WHITELIST_URLS)
        if os.path.exists(WHITELIST_FILE):
            with open(WHITELIST_FILE, "r") as f:
                content = f.read()
                if "," in content:
                    urls = content.split(",")
                else:
                    urls = content.splitlines()

                file_urls = {
                    url.strip()
                    for url in urls
                    if url.strip()
                }
                updated_whitelist.update(file_urls)
                LOADED_WHITELIST_URLS = updated_whitelist
        else:
            print(f"WARNING: Whitelist file not found at {WHITELIST_FILE}")
    except Exception as e:
        print(f"Error reloading whitelist: {e}", file=sys.stderr)
```

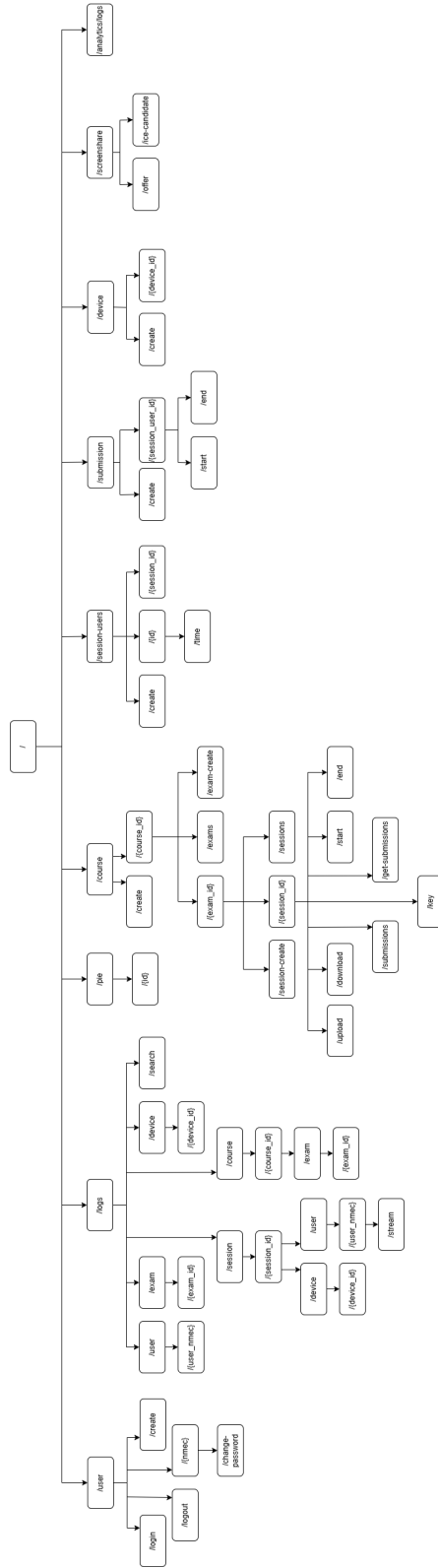
```

def request(flow: http.HTTPFlow) -> None:
    reload_whitelist()
    url = flow.request.pretty_url
    referer = flow.request.headers.get("Referer")
    if url in LOADED_WHITELIST_URLS:
        allowed_referers.add(url) # Allow dependent resources from this page
        return
    if is_whitelisted_domain(url):
        return
    if referer and referer in allowed_referers:
        return
    flow.response = http.Response.make(
        403, # Forbidden
        b"Access to url blocked by PIE.",
        {"Content-Type": "text/plain"}
    )

def response(flow: http.HTTPFlow) -> None:
    if flow.response.status_code in [301, 302, 303, 307, 308]: # Redirects
        location = flow.response.headers.get("Location")
        if location:
            full_redirect_url = urljoin(flow.request.url, location)
            if full_redirect_url not in LOADED_WHITELIST_URLS and not
                ↪ is_whitelisted_domain(full_redirect_url):
                flow.response = http.Response.make(
                    403,
                    b"Access denied: Redirect to unauthorized URL.",
                    {"Content-Type": "text/plain"}
                )
            return
    reload_whitelist()

```

A.3 ENDPOINTS TREE



A.4 DOCKER COMPOSE CONFIGURATION

```
version: "3.8"

services:
  app:
    build: .
    container_name: pie_api
    depends_on:
      - db
      - redis
      - elasticsearch
    env_file:
      - .env
    environment:
      - TZ=Europe/Lisbon
    ports:
      - "8000:8000"
    volumes:
      - ./app:/app
    command: ["uvicorn", "app.api.main:app", "--host", "0.0.0.0", "--port",
"8000", "--reload"]

  db:
    image: mysql:8.0
    container_name: mysql_container
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
      TZ: Europe/Lisbon
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  redis:
    image: redis:7
    container_name: redis_broker
    restart: unless-stopped
    environment:
      - TZ=Europe/Lisbon
    ports:
      - "6379:6379"

  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    container_name: phpmyadmin_container
    restart: always
```

```

environment:
  PMA_HOST: db
  MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
ports:
  - "8080:80"

clamav:
  image: clamav/clamav:stable
  container_name: clamav_container
  ports:
    - "3310:3310"
  volumes:
    - clamav_data:/var/lib/clamav
  restart: unless-stopped

elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:7.17.9
  container_name: elasticsearch
  environment:
    - discovery.type=single-node
    - bootstrap.memory_lock=true
    - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
    - TZ=Europe/Lisbon
  volumes:
    - elasticsearch_data:/usr/share/elasticsearch/data
  ports:
    - "9200:9200"
  restart: always

kibana:
  image: docker.elastic.co/kibana/kibana:7.17.9
  container_name: kibana
  environment:
    - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
  ports:
    - "5601:5601"
  depends_on:
    - elasticsearch
  restart: unless-stopped

volumes:
  mysql_data:
  app_files:
  clamav_data:
  elasticsearch_data:

```